

CALENDRIA

Assistant de Réservation Intelligent Multi-Canal avec IA

Rapport de Projet	Livrable Final
Auteur	BOUGHERARA Safi
Formation	CDA - Concepteur Développeur d'Applications
Promotion	2025-2026
Commanditaire	M. Gilles MOREL - « Le Gourmet Parisien »
Version	2.0 - Tag Git : v1.0
Date	Juillet 2026
Production	https://frontend-production-8a43.up.railway.app

Table des Matières

Section	Titre
I	Présentation personnelle et du projet (en anglais)
II	Présentation du projet CALENDRIA
III	Cahier des charges
IV	Méthodologie et organisation
V	Conception UI/UX
VI	Modélisation de la base de données
VII	Conception de l'application (UML)
VIII	Architecture multi-couches
IX	Sécurité et conformité
X	Politique de tests
XI	Déploiement et mise en production
XII	Veille technologique
XIII	Veille sécurité
XIV	Difficultés rencontrées et solutions
XV	Conclusion et perspectives
XVI	Remerciements
XVII	Annexes

Section I - Personal Introduction & Project Overview (in English)

1. About the Developer

My name is **Safi BOUGHERARA**, and I am currently enrolled in the **CDA (Concepteur Développeur d'Applications)** training program, promotion 2025-2026. This report is the final deliverable of my training, representing six months of full-stack web development work combining modern software engineering practices with artificial intelligence integration.

My professional interests lie at the intersection of backend architecture, frontend user experience, and AI-driven automation. CALENDRIA was designed to reflect all three of these interests in a single, production-ready application.

2. Project Summary

CALENDRIA is an AI-powered, multi-channel restaurant reservation management system. It was designed to address a concrete business problem faced by independent restaurateurs: the time-consuming process of managing phone reservations, estimated at up to two hours per day for a typical restaurant with 40 covers.

Core Features Delivered

- **AI Chatbot (Google Gemini 2.5 Flash)** - Understands natural language reservation requests, extracts key information (date, time, party size, name), checks availability in real time, and confirms bookings automatically
- **Multi-channel Access** - Web chatbot widget (embeddable via iframe), Telegram Bot (webhook-based), and Retell AI voice agent (phone reservations)
- **Smart Table Allocation Algorithm** - Assigns the most appropriate table (matching capacity) and proposes alternative time slots when the requested slot is unavailable
- **Restaurateur Dashboard (React SPA)** - Real-time reservation management, interactive floor plan with time slider, full CRUD operations, dark mode
- **Secure REST API (Symfony 6.4)** - JWT RS256 authentication, rate-limited login, CORS enforcement, OWASP-compliant endpoints
- **Production Deployment** - Fully containerized (Docker + nginx + PHP-FPM), deployed on Railway with automatic CI/CD via GitHub Actions

Technology Stack

Layer	Technology	Version	Role
Backend API	Symfony	6.4 LTS	REST API, business logic, security
Language	PHP	8.4	Server-side scripting
Frontend	React + TypeScript	18 / 5.x	SPA dashboard, chatbot widget
Build Tool	Vite	5.x	Frontend bundling
Database	PostgreSQL	15	Relational data persistence
ORM	Doctrine	3.x	Object-relational mapping

Layer	Technology	Version	Role
AI Engine	Google Gemini	2.5 Flash	NLP chatbot, intent extraction
Voice Agent	Retell AI	-	Phone reservation via webhook
Messaging	Telegram Bot API	-	Text-based reservation channel
Auth	LexikJWT (RS256)	2.x	Stateless JWT authentication
Containerization	Docker + Compose	24+	Dev & production environment
CI/CD	GitHub Actions	-	Automated tests + deployment
Hosting	Railway	-	PaaS cloud deployment

Key Performance Indicators

Indicator	Value
Functional coverage vs. initial specification	82% (14/17 features)
Automated tests (PHPUnit)	12 tests, 0 failures
Backend code (PHP)	~3,400 lines
Frontend code (TypeScript/TSX)	~3,200 lines
API endpoints	24 routes
Persisted entities	6 tables
Booking channels delivered	3 (web widget, Telegram, voice)

3. CDA Certification Context

This project was developed as the capstone of the **CDA (Concepteur Développeur d'Applications)** certification, a French professional qualification at RNCP Level 6 (equivalent to a Bachelor's degree). The certification validates the ability to design, develop, test, secure, and deploy complex web applications using modern software engineering practices.

The project spans six months (January to July 2026) and was structured around six monthly milestones (jalons), each producing a concrete deliverable:

- **Jalon 1** - Functional specification (CDCF)
- **Jalon 2** - Methodology & UI/UX mockups
- **Jalon 3** - Data modeling (MCD/MLD/MPD)
- **Jalon 4** - Software architecture & UML diagrams
- **Jalon 5** - Implementation & testing
- **Jalon 6** - Deployment, documentation & final report

Production URL : <https://frontend-production-8a43.up.railway.app>

GitHub Repository : https://github.com/SafiBougherara/projet_fin_annee

Section II - Présentation du Projet CALENDRIA

1. Contexte Métier

1.0 Identification du Commanditaire

Note : Dans le cadre de ce projet de formation, le commanditaire est fictif. Il représente le profil-type du client visé par la solution CALENDRIA.

Commanditaire	M. Gilles MOREL, Gérant de « Le Gourmet Parisien »
Établissement	Restaurant gastronomique, 10 tables, 40 couverts maximum
Adresse	15 rue de la Paix, 75001 Paris
Email de contact	gilles.morel@legourmetparisien.fr
Représentant pédagogique	Formateur CDA – [Nom du centre de formation]
Rôle dans le projet	Validation fonctionnelle, retours sur les maquettes, recette finale

Besoin exprimé : M. Morel passe en moyenne **2 heures par jour** à gérer les appels téléphoniques de réservation, dont une partie arrive en dehors des heures de service (répondeur non géré). Il souhaite automatiser ce processus tout en conservant la maîtrise complète de son planning via un tableau de bord web.

1.1 Domaine d'Activité

Secteur : Restauration / Hôtellerie-Restauration (CHR)

Cible : Restaurants indépendants, brasseries, bistrots, restaurants gastronomiques

1.2 Problématique Identifiée

Les restaurateurs font face à plusieurs défis dans la gestion de leurs réservations :

Problèmes Actuels

- **Charge téléphonique élevée :** Le personnel en salle doit gérer les appels de réservation pendant le service, perturbant l'expérience client
- **Disponibilité limitée :** Impossibilité de prendre des réservations en dehors des heures d'ouverture (matin, après-service, jours de fermeture)
- **Perte de clients :** Les clients qui n'obtiennent pas de réponse immédiate réservent chez un concurrent
- **Erreurs de réservation :** Risques de double réservation, mauvaise estimation du nombre de couverts, oubli de rappel
- **Gestion complexe des tables :** Difficulté à optimiser le remplissage (tables de 2, 4, 6, 8 personnes)
- **No-show :** Clients qui ne viennent pas sans prévenir, entraînant une perte de chiffre d'affaires
- **Coût du personnel :** Nécessité d'avoir du personnel dédié à la prise de réservations

1.3 Solution Proposée : CALENDRIA

CALENDRIA est un **assistant de réservation virtuel basé sur l'intelligence artificielle**, conçu pour automatiser la gestion des réservations de tables dans les restaurants.

Fonctionnement Global

1. **Client initie la réservation** : Via l'un des canaux disponibles : -  **Widget Web** : Chatbot intégré sur le site du restaurant -  **Telegram Bot** : Message vers le bot dédié -  **Agent Vocal IA (Retell AI)** : Appel téléphonique géré par IA
2. **Conversation IA** : Le chatbot engage une conversation naturelle et collecte : nom du client, numéro de téléphone, nombre de personnes, date et heure souhaitées, demandes spéciales (optionnel)
3. **Vérification de disponibilité en temps réel** : consultation des tables disponibles selon leur capacité, vérification des créneaux horaires configurés, application de l'algorithme d'attribution intelligent
4. **Validation automatique** :  **Si disponible** : Réservation confirmée instantanément  **Si indisponible** : Proposition automatique de créneaux alternatifs
5. **Dashboard restaurateur** : visualisation en temps réel de la nouvelle réservation

1.4 Utilisateurs Cibles

Utilisateurs Principaux

1. **Restaurateurs / Gérants** - Configurent les paramètres du restaurant (tables, horaires, services) - Consultent les réservations du jour/semaine - Peuvent annuler ou modifier une réservation - Analysent les statistiques de remplissage
2. **Clients** - Réserve via Widget Web, Telegram ou agent vocal - Reçoivent des confirmations instantanées - Peuvent annuler ou modifier via le même canal

Utilisateurs Secondaires

3. **Personnel de salle** (optionnel) - Consultent les réservations du jour, marquent les clients comme "arrivés" ou "no-show"

1.5 Valeur Ajoutée

- **Disponibilité 24/7** : Les clients peuvent réserver à tout moment, même à 2h du matin
- **Gain de temps** : Réduction estimée de **70%** du temps consacré à la gestion des appels (*source : étude OpenTable 2023*)
- **Réduction des erreurs** : Automatisation de la gestion des tables et couverts
- **Amélioration de l'expérience client** : Réponse immédiate 24h/7j, pas d'attente en ligne
- **Optimisation du remplissage** : Algorithme intelligent pour maximiser l'occupation des tables
- **Réduction des no-show** : Rappels automatiques et demande de confirmation (*objectif : réduire de 25 à 30%*)

Section III - Cahier des Charges

1. Objectifs du Projet

Les objectifs de CALENDRIA sont définis selon la méthode **SMART** :

1.1 Objectifs SMART

Critère	Description
Spécifique	Développer un assistant virtuel multi-canal capable de gérer automatiquement 90% des réservations de tables pour un restaurant via Widget Web, Telegram et agent vocal
Mesurable	Réduire le temps de gestion des réservations de 70% - Taux de complétion chatbot > 95% - Validation automatique instantanée - Réduction des no-show de 25 à 30%
Atteignable	Utilisation de technologies éprouvées (Symfony, React, PostgreSQL) avec un périmètre fonctionnel réaliste pour 6 mois
Réaliste	MVP chatbot fonctionnel pour 1 restaurant, développement séquentiel par jalon
Temporellement défini	Livraison finale : 30 juin 2026 (6 mois de développement avec jalons mensuels)

1.2 Objectifs Fonctionnels

1. **Automatiser les réservations** : Permettre aux clients de réserver une table sans intervention humaine
2. **Optimiser le remplissage** : Maximiser l'occupation des tables en proposant les créneaux optimaux
3. **Centraliser les réservations** : Regrouper toutes les réservations dans une interface unique
4. **Réduire les no-show** : Système de rappels et confirmations automatiques
5. **Simplifier la gestion** : Interface intuitive pour le restaurateur

1.3 Objectifs Techniques

1. **Respecter le cahier des charges technique** : Utiliser la stack imposée (Symfony, React, SQL, Docker, CI/CD)
2. **Garantir la sécurité** : Conformité RGPD et normes OWASP Top 10
3. **Assurer la qualité** : Couverture de tests > 80%, respect des normes de développement
4. **Faciliter le déploiement** : Application conteneurisée et déployable en 1 commande

2. Périmètre Fonctionnel

2.1 Fonctionnalités Principales (IN SCOPE)

2.1.1 Module IA Conversationnel - Multi-Canal

Architecture Partagée : Tous les canaux utilisent la même API REST Symfony, même logique de chatbot (Gemini 2.5 Flash), même algorithme de disponibilité.

Canal 1 : Widget Web (Priorité 1)

Chatbot intégrable sur le site du restaurant via iframe. Conversation naturelle en français, collecte d'informations, vérification de disponibilité, validation automatique.

Canal 2 : Bot Telegram (Priorité 2)

Chatbot via messagerie Telegram. Webhook `/api/chatbot/telegram`, gestion de session, confirmation automatique. Remplace WhatsApp Business (initialement prévu) sans barrière de paiement.

Canal 3 : Agent Vocal IA - Retell AI (Bonus)

Réservation par téléphone via agent vocal IA. Webhook `/api/chatbot/call`, messages d'erreur intelligents avec alternatives horaires.

2.1.2 Algorithme de Gestion des Tables

- **Attribution intelligente** : Priorité aux tables de capacité exacte, puis table immédiatement supérieure
- **Calcul de disponibilité** : Vérification en temps réel selon les services configurés
- **Alternatives horaires** : Propositions automatiques par pas de 30 min si créneau indisponible
- **Gestion des contraintes** : Délai minimum/maximum, blocage de dates, fermetures exceptionnelles

2.1.3 Interface Restaurateur (Dashboard Web)

Application React SPA avec : authentification JWT, tableau de bord temps réel, plan de salle interactif avec slider temporel, gestion CRUD des réservations, gestion des tables et services, fiche client avec historique, mode sombre persistant.

2.2 Fonctionnalités Hors Scope (OUT OF SCOPE)

- ❌ Paiement en ligne
- ❌ Commande de plats
- ❌ Application mobile native
- ❌ Multi-langues (français uniquement)
- ❌ Multi-restaurants dans le MVP
- ❌ Intégration avec logiciels de caisse
- ❌ Système de fidélité ou d'avis clients

3. Exigences Techniques**3.1 Stack Technique****Back-end**

Composant	Technologie	Version	Justification
Framework	Symfony	6.4 LTS	Framework PHP imposé, version LTS pour stabilité

Composant	Technologie	Version	Justification
Langage	PHP	8.4	Dernière version stable, typage strict
ORM	Doctrine	3.x	Intégré à Symfony, prévention injections SQL
Authentification	LexikJWTAuthenticationBundle	2.x	Tokens JWT pour API stateless

Front-end

Composant	Technologie	Version	Justification
Framework	React	18+	Framework JavaScript imposé, écosystème riche
Langage	TypeScript	5.x	Typage statique, meilleure maintenabilité
Build tool	Vite	5.x	Build rapide, Hot Module Replacement

Base de Données

Composant	Technologie	Justification
SGBD	PostgreSQL 15	Meilleure gestion des contraintes, type JSON natif, performances
Modélisation	Méthode MERISE	Imposée par le CDC (MCD/MLD/MPD)
Normalisation	3NF	Forme normale 3 pour éviter les redondances

Containerisation & DevOps

Composant	Technologie	Justification
Conteneurs	Docker 24+	Imposé par le CDC, environnement uniforme
Orchestration	Docker Compose 2.x	Gestion multi-conteneurs (API, BDD, Front)
CI/CD	GitHub Actions	Automatisation tests/déploiement, intégré GitHub
Hébergement	Railway	PaaS supportant Docker, déploiement continu

3.2 Sécurité et Conformité

Menace	Protection Implémentée
Injection SQL	ORM Doctrine avec requêtes préparées, paramètres liés
XSS	Échappement automatique React/Twig
CSRF	API stateless avec JWT (pas de sessions/cookies)
Brute force	Rate limiting 5 tentatives / 15 min par IP
Mots de passe	Argon2id (algorithme Symfony par défaut)
RGPD	Consentement explicite, droit à l'oubli, données minimales

4. Contraintes et Enjeux

4.1 Contraintes Temporelles

Durée totale : 6 mois (janvier à juin 2026)

Jalon	Mois	Livrable
1	Janvier 2026	CDCF
2	Février 2026	Méthodologie + Maquettes UI/UX
3	Mars 2026	MCD/MLD/MPD + API REST
4	Avril 2026	Chatbot + Dashboard + UML
5	Mai 2026	Widget Web + Tests + Sécurité
6	Juin 2026	Déploiement + Documentation finale

4.2 Risques Identifiés et Mitigation

Risque	Probabilité	Impact	Mitigation
Dépassement crédit API	Faible	Faible	Google Gemini Flash gratuit en tier 0 + Telegram gratuit
Complexité algorithme tables	Moyenne	Élevé	Mocks de scénarios, buffer turnover configurable, tests exhaustifs
Retard sur planning	Faible	Moyen	Développement anticipé, Widget/SMS optionnels si retard
Bugs en production	Moyenne	Moyen	Tests automatisés > 80%, tests manuels avant chaque jalon

5. Critères de Succès

5.1 Critères Fonctionnels

- Au minimum **2 canaux fonctionnels**  (Widget Web + Telegram + Retell AI)
- Taux de complétion > **95%**  (dialogue structuré en questions fermées)
- Calcul correct de disponibilité dans **100%** des cas 
- Validation automatique instantanée 

5.2 Critères Techniques

- Couverture de tests > **80%** des classes métier 
- Application déployable via Docker en **1 commande** 
- Pipeline CI/CD fonctionnelle à chaque push 
- Conformité OWASP Top 10 

Section IV - Méthodologie et Organisation

1. Méthode de Gestion de Projet

1.1 Méthodologie Retenue : une Méthode Agile Adaptée au Travail Seul

Choix : démarche itérative inspirée de Scrum, et non Scrum à la lettre : Scrum suppose une équipe, ce projet a été mené par un développeur unique.

Justification du Choix

Critère	Cycle en V	Kanban	Agile itératif	Choix
Flexibilité	Rigide	Très flexible	Flexible avec cadre temporel	✓ Agile
Correspondance jalons	Bon	Pas de jalons naturels	Sprints = Jalons mensuels	✓ Agile
Gestion des risques	Détection tardive	Correcte	Détection précoce (revues de jalon)	✓ Agile
Suivi d'avancement	Jalons séquentiels	Flux continu	Incréments livrables à chaque sprint	✓ Agile

Pratiques conservées et pratiques abandonnées

Pratique	Application dans le projet
Sprint	1 mois = 1 jalon — 6 sprints de janvier à juin
Backlog priorisé	Liste de fonctionnalités classées en MoSCoW, figée au Jalon 1 et révisée à chaque jalon
Incrément livrable	Chaque jalon produit un livrable démontrable
Revue de sprint	Présentation au référent à la fin de chaque jalon
Rétrospective	Bilan écrit en fin de jalon — c'est elle qui a déclenché le passage de WhatsApp Business à Telegram
Suivi quotidien	Historique Git, commits typés selon Conventional Commits

Les **rôles** (Product Owner, Scrum Master) et la **mêlée quotidienne** ont été volontairement abandonnés : ce sont des rituels de coordination d'équipe, qui n'ont aucun destinataire lorsque l'on travaille seul.

1.2 Backlog Produit (MoSCoW)

Must Have (Obligatoire)

- En tant que **client**, je veux réserver une table via le chatbot pour ne pas avoir à appeler le restaurant
- En tant que **restaurateur**, je veux voir toutes les réservations du jour sur un dashboard

- En tant que **restaurateur**, je veux configurer mes tables et horaires
- En tant que **client**, je veux recevoir une confirmation immédiate après ma réservation

Should Have (Important)

- En tant que **client**, je veux accéder à plusieurs canaux de réservation
- En tant que **restaurateur**, je veux consulter les statistiques de remplissage

Won't Have (Hors scope)

- Paiement en ligne, commande de plats, multi-langues, application mobile native

2. Gestion du Code Source (Git)

2.1 Stratégie de Branches - Trunk-Based Development

Le projet a été mené par un développeur unique. Git Flow existe pour **synchroniser plusieurs développeurs** et permettre la revue croisée ; ouvrir des branches `feature/*` que je serais le seul à créer, relire et fusionner n'aurait été qu'une cérémonie sans bénéfice. J'ai donc retenu le **trunk-based development** : des commits petits et fréquents sur `main`, la **CI bloquante** tenant le rôle du relecteur.

```
main ← unique branche de travail et de livraison
|
| 79 commits · CI GitHub Actions déclenchée à chaque push
|
├─ tag v0.9-beta   Jalon 5 : plan de salle, tests, mode sombre
|
└─ tag v1.0       Livraison finale
```

Indicateur	Valeur
Branches	1 (<code>main</code>)
Commits	79
Commits conformes à Conventional Commits	59 (75 %)
Tags de release	<code>v0.9-beta</code> , <code>v1.0</code>

Limite assumée : en équipe, cette stratégie ne tiendrait pas. Je passerais à des branches de fonctionnalité avec Pull Request obligatoire et revue par un pair avant fusion.

2.2 Convention de Commits

Les messages de commit suivent la convention **Conventional Commits** :

```
<type>(<scope>): <description>
```

Types :

- feat: Nouvelle fonctionnalité
- fix: Correction de bug
- docs: Documentation
- refactor: Refactoring du code
- test: Ajout/modification de tests
- chore: Tâches de maintenance

Exemples :

- feat(api): Add reservation CRUD endpoints
- fix(chatbot): Fix date parsing for French format
- test(api): Add unit tests for table algorithm

2.3 Historique du Projet

- Structure du projet (backend / frontend / docker / documentation)
- Backend Symfony 6.4 avec entités Doctrine et migrations versionnées
- API REST exposée par 24 routes réparties en 4 contrôleurs
- Authentification JWT (LexikJWTAuthenticationBundle)
- Frontend React 19 + TypeScript + Vite
- Docker Compose (PostgreSQL, backend, frontend)

3. CI/CD Planifié et Réalisé

3.1 Pipeline CI (GitHub Actions)

Le fichier `.github/workflows/ci.yml` déclenche **trois jobs parallèles** à chaque `push` et à chaque Pull Request. Aucun merge sur `main` n'est accepté si l'un d'eux échoue.



3.2 Job de sécurité : dépendances et secrets

Un pipeline qui ne vérifie que la compilation laisse passer deux familles d'incidents très courantes : l'introduction d'une **dépendance vulnérable** et le **commit accidentel d'un secret**. Un troisième job a donc été ajouté pour couvrir ces deux risques.

Contrôle	Outil	Risque couvert	Comportement
Dépendances PHP	<code>composer audit --locked</code>	CVE connues dans les paquets Composer	Bloquant
Dépendances JavaScript	<code>npm audit --audit-level=high</code>	CVE connues dans les paquets npm	Bloquant à partir du niveau « high »
Secrets commités	<code>gitleaks detect</code>	Token, clé d'API ou identifiant poussé dans le dépôt	Bloquant

Gitleaks analyse **l'intégralité de l'historique Git**, et pas seulement le dernier commit : un secret supprimé dans un commit ultérieur reste récupérable et doit donc être détecté. La configuration

`.gitleaks.toml` étend le jeu de règles par défaut avec les secrets propres au projet, et déclare en liste blanche les valeurs factices des fichiers d'exemple afin d'éviter les faux positifs.

```
[[rules]]
id = "telegram-bot-token"
description = "Token de bot Telegram"
regex = '''[0-9]{8,12}:AA[A-Za-z0-9_-]{32,}'''

[[rules]]
id = "google-gemini-api-key"
description = "Clé d'API Google (Gemini)"
regex = '''AIza[0-9A-Za-z_-]{35}'''
```

Résultat immédiat : dès sa première exécution, le job a signalé **4 vulnérabilités** dans les dépendances du frontend, dont **3 de sévérité haute** sur `react-router` (redirection ouverte, XSS, déni de service par appariement de routes inefficace). Elles ont été corrigées par montée de version compatible, puis le build a été revalidé. Cet incident justifie à lui seul l'ajout du job : ces failles étaient présentes en production sans qu'aucun signal ne les révèle.

Le nettoyage des variables d'environnement d'exemple a été mené dans le même mouvement :

`TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN` et `OPENAI_API_KEY` subsistaient dans `.env.example` alors que ces services ne sont plus utilisés. Une variable morte est une invitation à y placer un jour une vraie valeur.

3.3 Outils de Suivi

Activité	Fréquence	Support
Suivi de l'avancement	Quotidien	Historique Git (messages de commit typés)
Revue de sprint	Fin de mois	Présentation au référent
Rétrospective	Fin de mois	Bilan écrit intégré au livrable du jalon

Section V - Conception UI/UX

1. Sitemap / Architecture des Écrans

1.1 Plan du Site – Dashboard Restaurateur

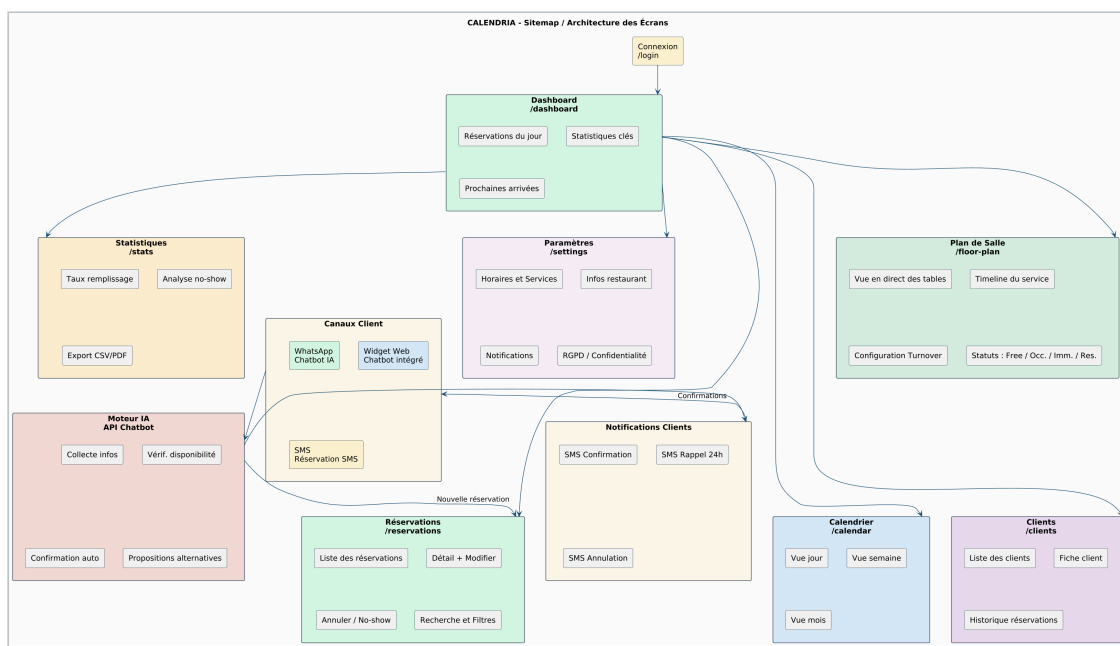


Figure 5.1 - Sitemap : architecture des écrans du dashboard

2. Zoning

Le **zoning** est la première étape de la conception d'interface. Il découpe chaque écran en **zones fonctionnelles** (en-tête, navigation, contenu principal, zone d'action) **avant toute décision graphique**. Son objectif est de valider la hiérarchie de l'information avec le commanditaire sans que celui-ci soit distrait par l'esthétique : ni couleur, ni typographie, ni contenu réel à ce stade.

Trois écrans structurants ont été zonés : la page de connexion, le tableau de bord et l'écran de paramétrage du restaurant.

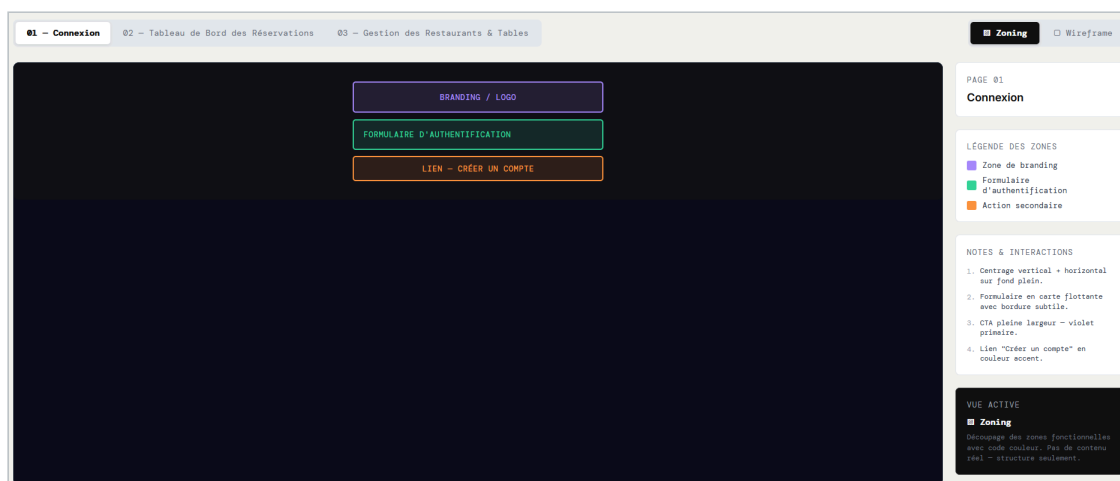


Figure 5.2 - Zoning : page de connexion

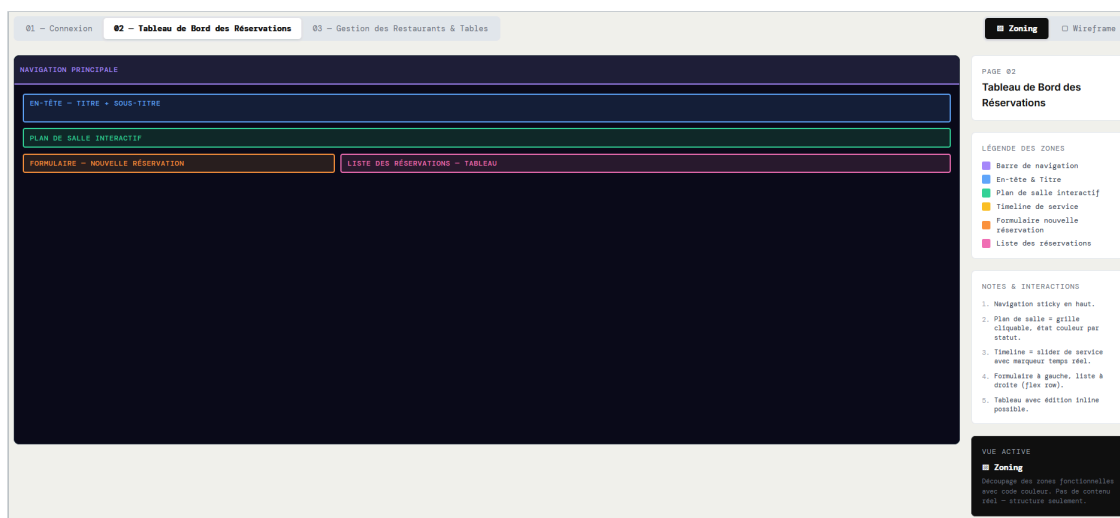


Figure 5.3 - Zoning : tableau de bord

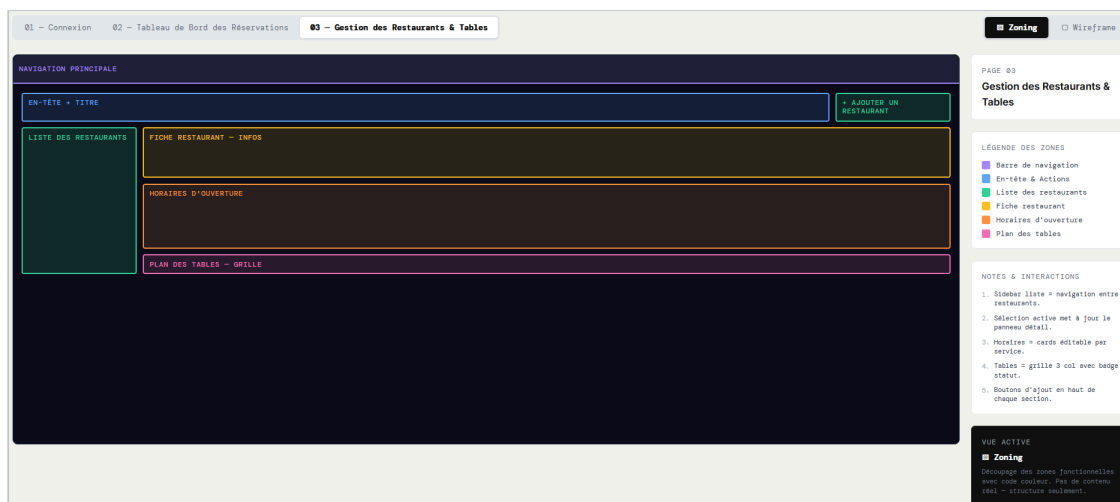


Figure 5.4 - Zoning : paramétrage du restaurant et des tables

3. Wireframes (Maquettes Fil de Fer)

Le **wireframe** précise le zoning : il place les composants réels (champs de formulaire, tableaux, boutons, indicateurs) et fixe la densité d'information, toujours en niveaux de gris pour rester centré sur la structure et non sur le style.

3.1 Page de Connexion

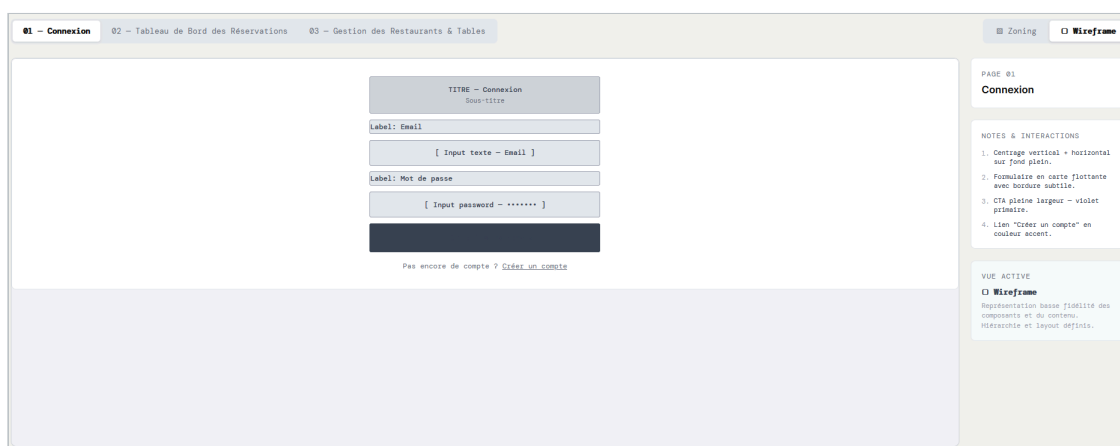


Figure 5.5 - Wireframe : page de connexion (email, mot de passe, lien d'inscription)

3.2 Tableau de Bord – Desktop

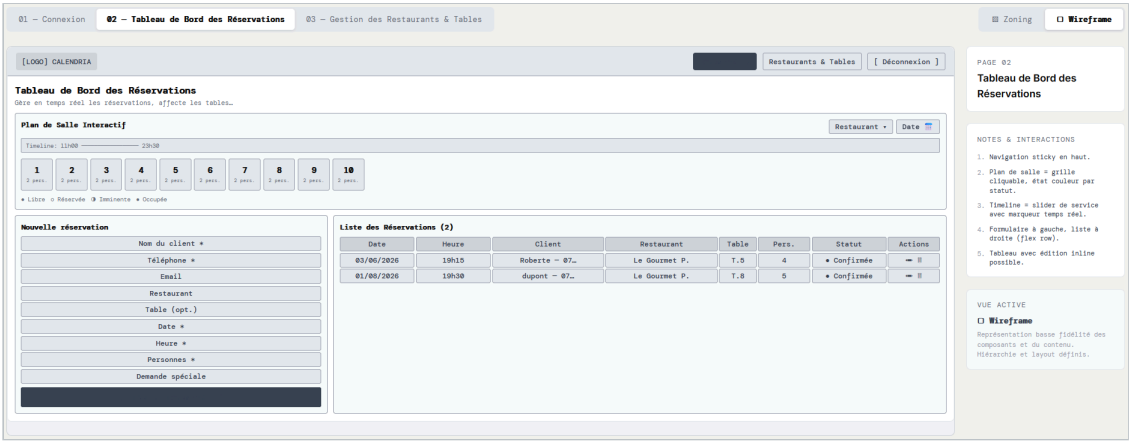


Figure 5.6 - Wireframe : tableau de bord (plan de salle, formulaire de réservation, liste des réservations)

3.3 Paramétrage du Restaurant

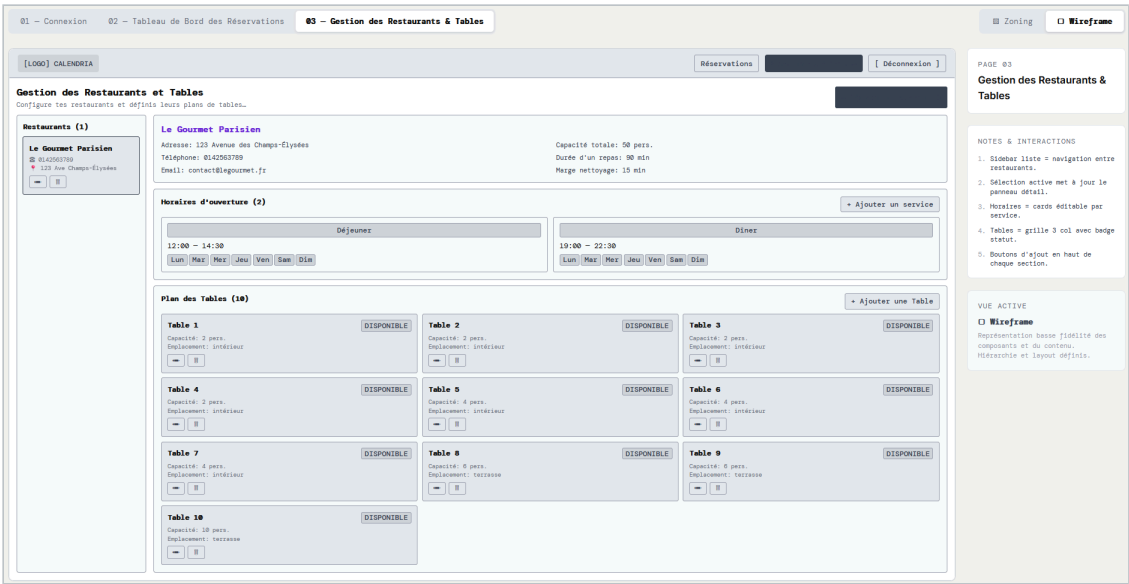
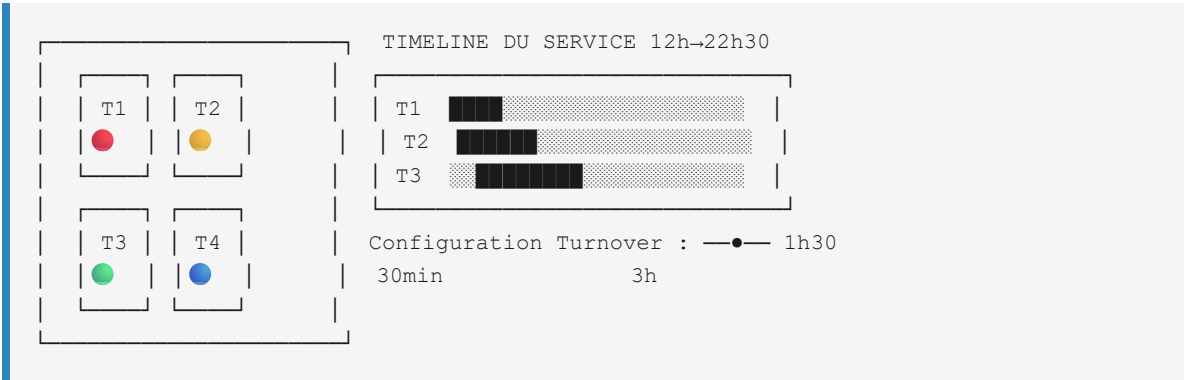


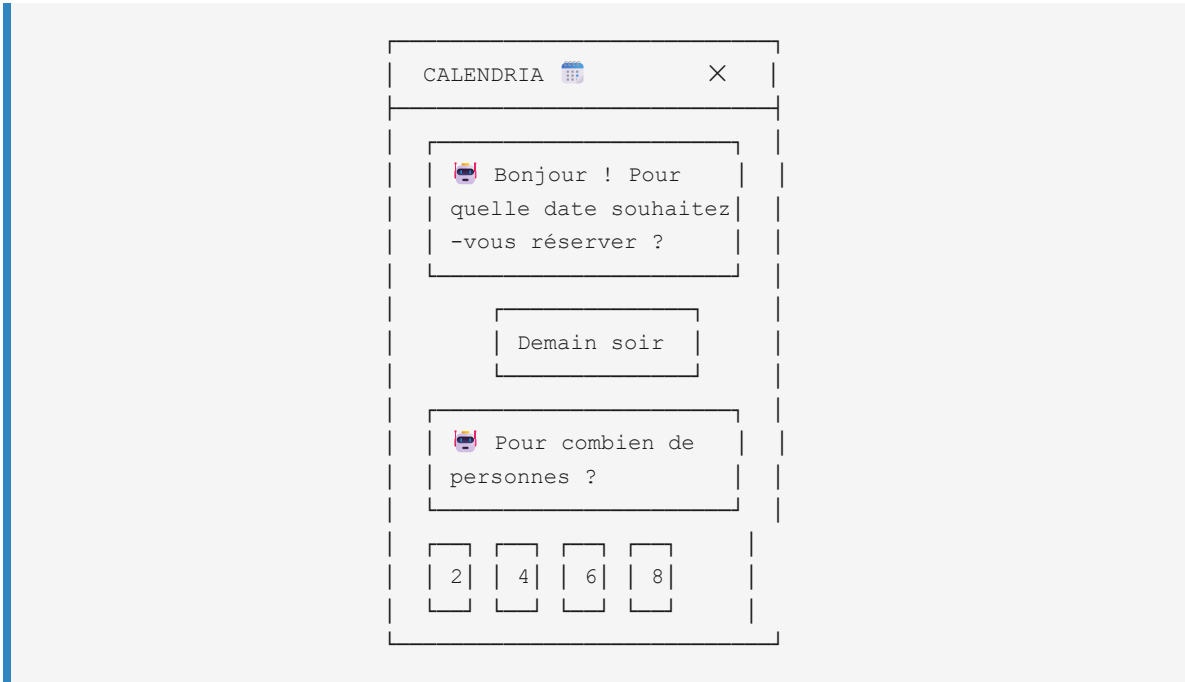
Figure 5.7 - Wireframe : gestion du restaurant, des tables et des services

3.4 Plan de Salle – Vue en Direct

Le plan de salle combine une vue spatiale des tables et une timeline du service. Cette double lecture a été validée dès le wireframe car elle conditionne toute l'ergonomie du dashboard.



3.5 Widget Web Chatbot (Côté Client)



4. Charte Graphique

4.1 Identité Visuelle

Nom : CALENDRIA | Ton : Professionnel, moderne et chaleureux

4.2 Palette de Couleurs

Couleurs Principales

Rôle	Nom	Hex	Usage
Primaire	Bleu Profond	#1A5276	Navigation, headers, boutons principaux
Primaire clair	Bleu Ciel	#2E86C1	Liens, éléments interactifs
Secondaire	Corail Chaud	#E74C3C	Accents, CTA secondaires, notifications
Fond sidebar	Bleu Nuit	#0E2F44	Barre latérale

Couleurs de Statut

Statut	Hex	Usage
 Confirmée / Disponible	#27AE60	Réservation confirmée
 Imminente	#F39C12	Arrivée dans < 30 min
 Arrivé	#3498DB	Client présent
 No-show	#E74C3C	Client absent
 Annulée	#7F8C8D	Réservation annulée

4.3 Typographie

Usage	Police	Famille
Titres (h1, h2, h3)	Outfit	Sans-serif (Google Fonts)
Texte courant	Inter	Sans-serif (Google Fonts)
Code / Données	JetBrains Mono	Monospace (Google Fonts)

5. Maquettes Haute Fidélité

Les maquettes haute fidélité ont été réalisées dans **Figma** en appliquant la charte graphique définie ci-dessus. Elles couvrent au minimum 2 écrans :

1. **Dashboard (Tableau de bord)** – Version Desktop (1440px)
2. **Dashboard (Tableau de bord)** – Version Mobile (375px)
3. Page de Connexion – Desktop
4. Page Réservations – Desktop
5. Widget Chatbot – Desktop

6. Considérations UX

6.1 Principes Appliqués

A. Mobile First

L'application est conçue Mobile First : le dashboard est d'abord pensé pour être consultable rapidement sur smartphone (le restaurateur vérifie ses réservations pendant le service). Navigation mobile via **bottom navigation bar**, desktop via **sidebar** latérale.

B. Simplicité d'Utilisation

- **3 clics maximum** pour accéder à n'importe quelle information importante
- **Dashboard par défaut** après connexion
- **Actions contextuelles** directement dans les listes

C. Code Couleur des Statuts

Un système de couleurs cohérent est utilisé dans toute l'application. Les informations ne sont jamais transmises uniquement par la couleur (ajout d'icônes et texte) - conformité WCAG 2.1 niveau AA.

6.2 Accessibilité (WCAG 2.1 AA)

Critère	Implémentation
Contraste	Ratio minimum 4.5:1 pour le texte
Navigation clavier	Tous les éléments interactifs accessibles
Taille de cible	Minimum 44x44px sur mobile

Critère	Implémentation
Focus visible	Indicateur de focus sur tous les éléments

6.3 Responsive Design

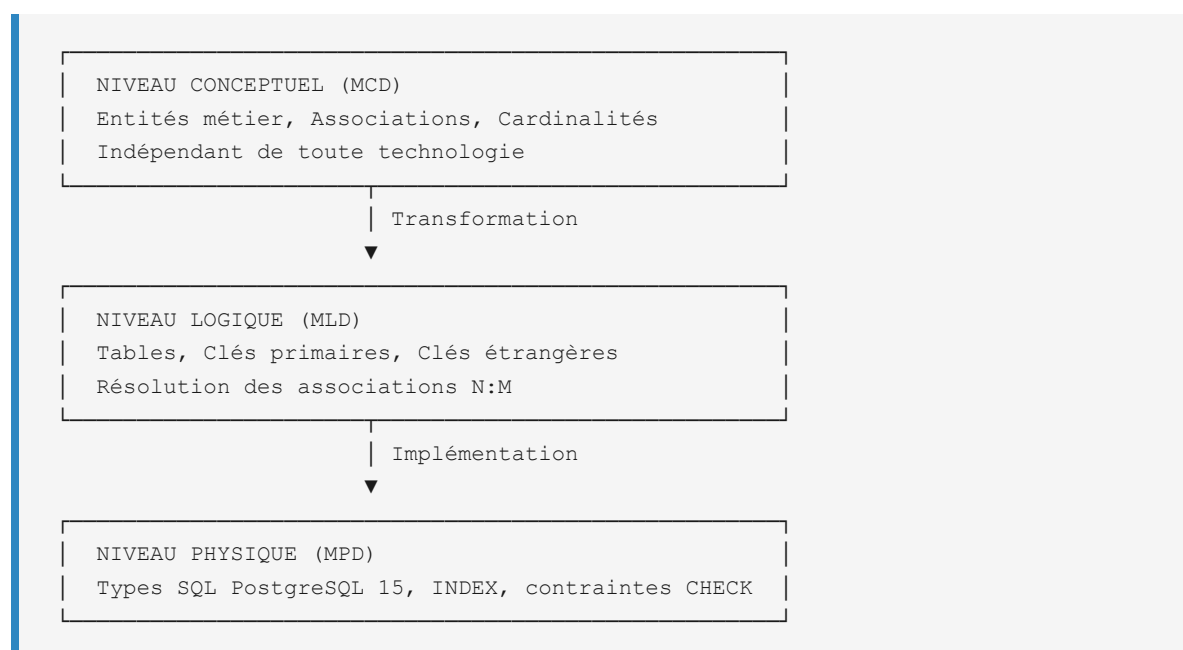
Breakpoint	Largeur	Adaptation
Mobile	< 768px	Navigation bottom bar, cards empilées, sidebar cachée
Tablet	768px - 1024px	Sidebar collapsible, grille 2 colonnes
Desktop	> 1024px	Sidebar étendue, grille multi-colonnes, tableau complet

Section VI - Modélisation de la Base de Données

1. Introduction à la Démarche MERISE

La modélisation de la base de données de CALENDRIA suit la démarche **MERISE**, une méthodologie de conception par niveaux qui garantit la cohérence entre les besoins fonctionnels et l'implémentation technique (base PostgreSQL 15).

Le respect strict de la séparation des niveaux est essentiel : on parle d'**entités**, de **propriétés** et d'**associations** au niveau conceptuel, et seulement de **tables**, de **colonnes** et de **clés** à partir du niveau logique. Les clés primaires et étrangères n'apparaissent donc jamais sur le MCD : elles sont le résultat de la transformation du MCD en MLD.



1.1 Formalisme retenu pour le MCD

Élément	Représentation	Remarque
Entité	Rectangle portant un nom au singulier	Contient ses propriétés ; l'identifiant est souligné
Association	Ovale portant un verbe à l'infinitif	Relie deux entités ou plus
Cardinalité	Couple (min,max) porté par chaque patte	Se lit du côté de l'entité concernée
Clé primaire / étrangère	Absente du MCD	Introduite au niveau logique (MLD) uniquement

2. Entités du Modèle (9 entités)

Le modèle conceptuel décrit la **cible fonctionnelle** du produit. Le MVP livré en implémente **6 entités sur 9** : les trois dernières correspondent à des fonctionnalités volontairement écartées du périmètre (voir section XV).

#	Entité	Rôle	Implémentée dans le MVP
1	RESTAURANT	Établissement racine	Oui
2	UTILISATEUR	Compte dashboard (authentification JWT)	Oui
3	TABLE_RESTAURANT	Table physique du restaurant	Oui
4	SERVICE	Période de service (midi/soir) + jours d'ouverture	Oui
5	CLIENT	Personne qui réserve	Oui
6	RESERVATION	Réservation de table (entité centrale)	Oui
7	FERMETURE_EXCEPTIONNELLE	Dates de fermeture ponctuelle	Non — hors périmètre
8	CONVERSATION_IA	Historique de conversation multi-canal	Non — sessions en cache Symfony
9	NOTIFICATION	Message de confirmation ou de rappel	Non — hors périmètre

3. Dictionnaire des Données

3.1 RESTAURANT

Attribut	Type	Contraintes	Obligatoire
id	Entier auto-incrémenté	PK	✓
nom	Chaîne (255)	NOT NULL	✓
adresse	Texte libre	NOT NULL	✓
telephone	Chaîne (20)	NOT NULL	✓
email	Chaîne (255)	NOT NULL	✓
capacite_totale	Entier positif	NOT NULL, > 0	✓
duree_repas	Entier positif (minutes)	NOT NULL, > 0	✓
buffer_nettoyage	Entier positif (minutes)	NOT NULL, ≥ 0	✓
created_at	TIMESTAMP	NOT NULL	✓

3.2 UTILISATEUR

Attribut	Type	Contraintes	Obligatoire
id	Entier auto-incrémenté	PK	✓
email	Chaîne (180)	NOT NULL, UNIQUE	✓

Attribut	Type	Contraintes	Obligatoire
roles	JSON (tableau)	NOT NULL	✓
password	Chaîne (255)	NOT NULL (hachage Argon2id)	✓

Le compte du dashboard n'est volontairement rattaché à aucun restaurant dans le MVP. Une clé étrangère obligatoire interdirait un gérant multi-établissements ainsi qu'un compte d'administration générale ; l'association sera ajoutée lors du passage en multi-restaurant.

3.3 TABLE_RESTAURANT

Attribut	Type	Contraintes	Obligatoire
id	Entier auto-incrémenté	PK	✓
restaurant_id	Référence → RESTAURANT	FK, NOT NULL	✓
numero_table	Chaîne (10)	NOT NULL	✓
capacite	Entier positif	NOT NULL, > 0	✓
type	Chaîne (50)	NOT NULL, ENUM: interieur/terrasse/vip/prive	✓
statut	Chaîne (20)	NOT NULL, ENUM: disponible/hors_service	✓
created_at	TIMESTAMP	NOT NULL	✓

3.4 SERVICE

Attribut	Type	Contraintes	Obligatoire
id	Entier auto-incrémenté	PK	✓
restaurant_id	Référence → RESTAURANT	FK, NOT NULL	✓
type	Chaîne (20)	NOT NULL, ENUM: midi/soir	✓
heure_debut	TIME	NOT NULL	✓
heure_fin	TIME	NOT NULL	✓
jours_ouverture	JSON	NOT NULL, ex: ["lundi", "mardi"]	✓
created_at	TIMESTAMP	NOT NULL	✓

3.5 CLIENT

Attribut	Type	Contraintes	Obligatoire
id	Entier auto-incrémenté	PK	✓
nom	Chaîne (255)	NOT NULL	✓
telephone	Chaîne (20)	NOT NULL	✓

Attribut	Type	Contraintes	Obligatoire
email	Chaîne (255)	nullable	✗
preferences	JSON	nullable	✗
consentement_rgpd	BOOLEAN	NOT NULL	✓
taux_noshow	DECIMAL	nullable, ≥ 0 , ≤ 1	✗
created_at	TIMESTAMP	NOT NULL	✓

`taux_noshow` est un **attribut dérivé** : il se recalcule à partir des réservations du client. Il est stocké pour éviter un recalcul à chaque affichage, ce qui constitue une dénormalisation assumée.

3.6 RESERVATION

Attribut	Type	Contraintes	Obligatoire
id	Entier auto-incrémenté	PK	✓
client_id	Référence → CLIENT	FK, NOT NULL	✓
restaurant_id	Référence → RESTAURANT	FK, NOT NULL	✓
table_id	Référence → TABLE_RESTAURANT	FK, nullable	✗
date_reservation	DATE	NOT NULL	✓
heure_reservation	TIME	NOT NULL	✓
nombre_personnes	Entier positif	NOT NULL, > 0	✓
statut	Chaîne (20)	NOT NULL : confirmée / arrivée / annulée / noshow	✓
demandes_speciales	Texte	nullable	✗
created_at	TIMESTAMP	NOT NULL	✓

`table_id` est nullable : une réservation peut exister avant qu'une table ne lui soit attribuée. Le champ `statut` est aujourd'hui une chaîne ; son passage en énumération PHP 8.1 est identifié comme amélioration.

3.7 FERMETURE_EXCEPTIONNELLE (modélisée, non implémentée dans le MVP)

Attribut	Type	Contraintes	Obligatoire
id	Entier auto-incrémenté	PK	✓
restaurant_id	Référence → RESTAURANT	FK, NOT NULL	✓
date_fermeture	DATE	NOT NULL	✓
motif	Chaîne (255)	nullable	✗
created_at	TIMESTAMP	NOT NULL	✓

3.8 CONVERSATION_IA (modélisée, non implémentée dans le MVP)

Attribut	Type	Contraintes	Obligatoire
id	Entier auto-incrémenté	PK	✓
restaurant_id	Référence → RESTAURANT	FK, NOT NULL	✓
client_id	Référence → CLIENT	FK, nullable	✗
telephone_appelant	Chaîne (20)	NOT NULL	✓
canal	ENUM	NOT NULL : widget / telegram / vocal	✓
date_debut	TIMESTAMP	NOT NULL	✓
date_fin	TIMESTAMP	nullable	✗
transcription	TEXT	nullable	✗
donnees_collectees	JSON	nullable	✗
reservation_id	Référence → RESERVATION	FK, nullable	✗
statut	ENUM	NOT NULL : complete / incomplete / erreur	✓
created_at	TIMESTAMP	NOT NULL	✓

`client_id` est nullable car la conversation démarre avant l'identification de l'appelant. `restaurant_id` est en revanche obligatoire : le webhook doit savoir pour quel établissement le chatbot répond. Dans le MVP, cet état de session est conservé dans le **cache Symfony** et n'est pas persisté.

3.9 NOTIFICATION (modélisée, non implémentée dans le MVP)

Attribut	Type	Contraintes	Obligatoire
id	Entier auto-incrémenté	PK	✓
reservation_id	Référence → RESERVATION	FK, NOT NULL	✓
destinataire	Chaîne (20)	NOT NULL	✓
message	TEXT	NOT NULL	✓
type	ENUM	NOT NULL : confirmation / rappel / annulation	✓
canal	ENUM	NOT NULL : sms / telegram / email	✓
statut_envoi	ENUM	NOT NULL : en_attente / envoye / echec	✓
date_envoi	TIMESTAMP	nullable	✗
id_externe	Chaîne (50)	nullable, UNIQUE si non null	✗
created_at	TIMESTAMP	NOT NULL	✓

L'entité a été renommée de **NOTIFICATION_SMS** en **NOTIFICATION** : le nom d'origine préjugait du canal d'envoi alors qu'une confirmation peut aussi partir par Telegram ou par courriel. Le canal devient une propriété, et l'identifiant externe n'est plus lié à un prestataire particulier.

4. Diagrammes MERISE

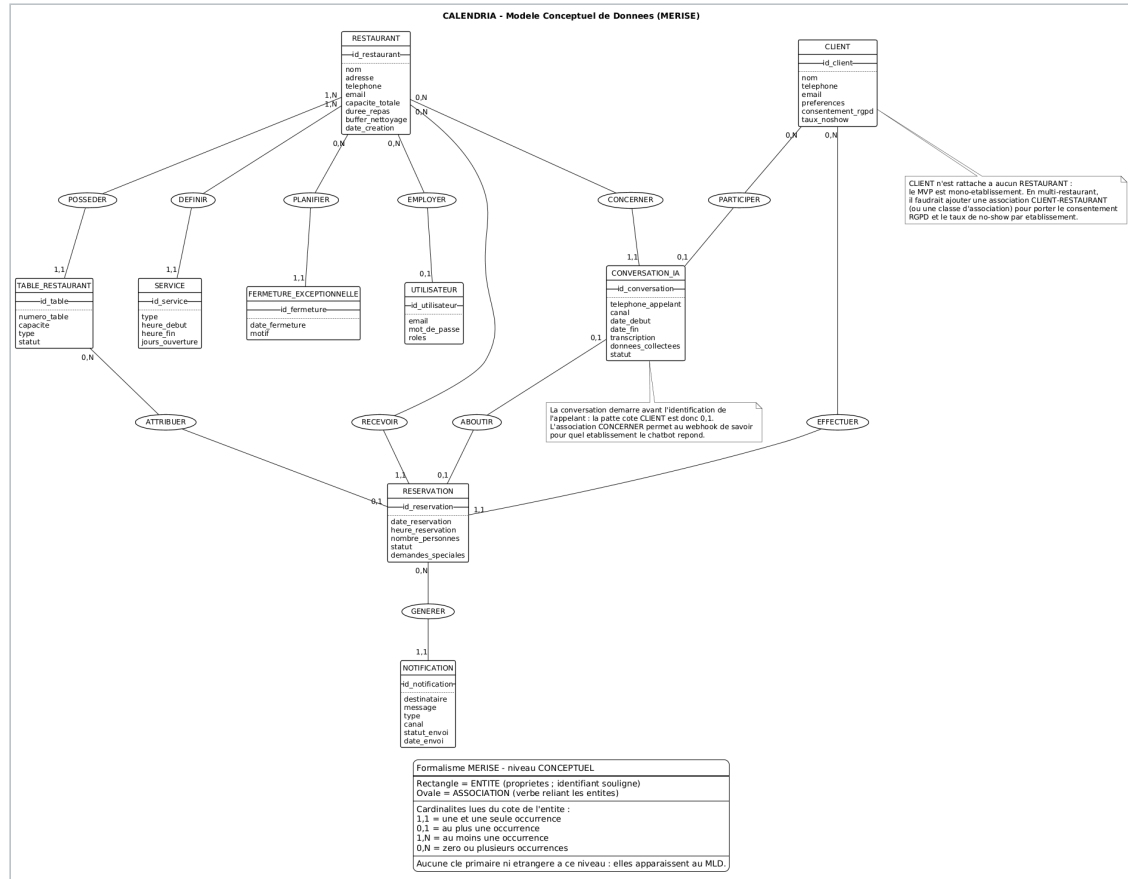


Figure 6.1 - Modèle Conceptuel de Données (MCD) : entités en rectangles, associations en ovales, cardinalités (min,max) sur chaque patte. Aucune clé n'apparaît à ce niveau.

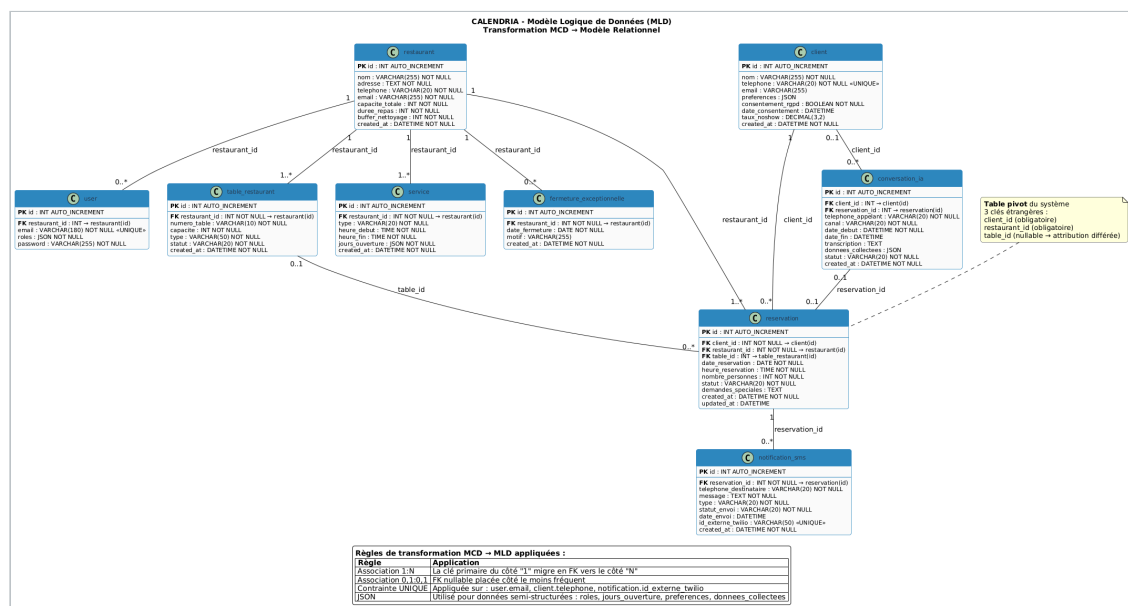


Figure 6.2 - Modèle Logique de Données (MLD) : les associations deviennent des tables et des clés étrangères

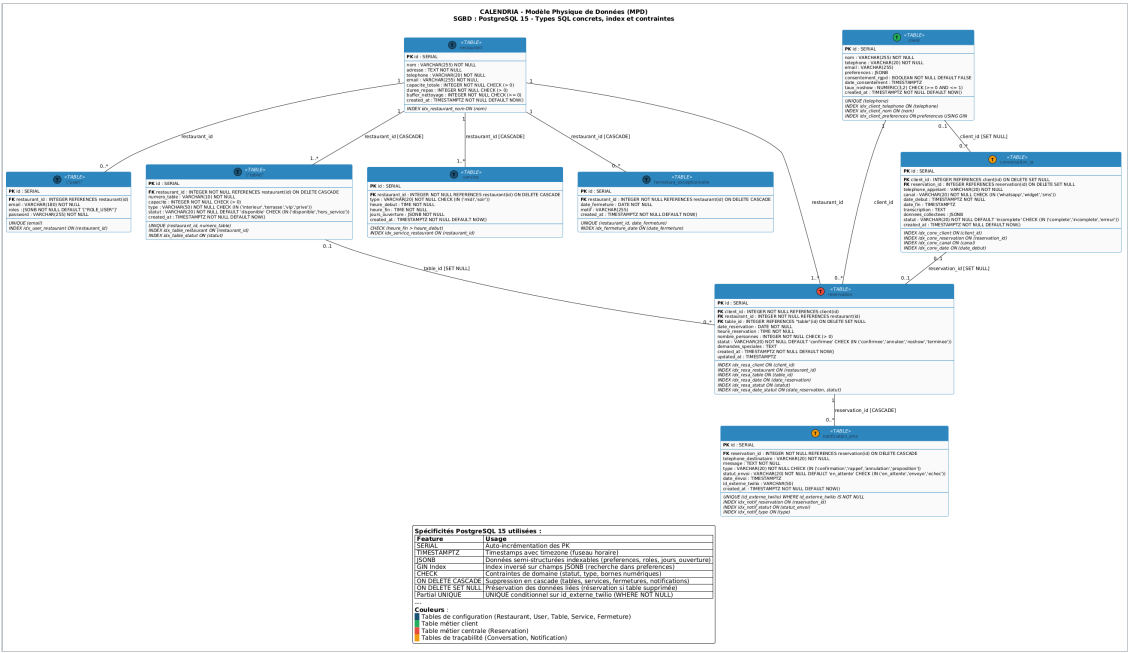


Figure 6.3 - Modèle Physique de Données (MPD) - PostgreSQL 15

Section VII - Conception de l'Application (UML)

Introduction

Cette section présente la conception technique de CALENDRIA. Elle fait le lien entre les besoins fonctionnels (CDCF), les parcours utilisateurs (zoning et wireframes) et le modèle de données (MERISE). Les diagrammes sont écrits en PlantUML, versionnés avec le code dans `documentation/Jalon4/`, et rendus en niveaux de gris pour rester lisibles à l'impression.

1. Vue Fonctionnelle (Use Cases)

Ce diagramme offre une vision panoramique des interactions entre les utilisateurs et le système. Il ne représente que les fonctionnalités **effectivement livrées** : les notifications SMS, les rappels à 24 h et les statistiques avancées, écartées du périmètre, n'y figurent pas.

- 3 **acteurs** : le Client, le Restaurateur et l'Assistant IA (acteur système)
- 2 **sous-systèmes** : « Réservation multi-canal » et « Dashboard restaurateur »
- Les associations **acteur / cas d'utilisation** sont des **traits simples sans pointe** : la flèche n'aurait de sens que pour indiquer une navigabilité, notion absente des cas d'utilisation
- Les dépendances `<<include>>` et `<<extend>>` sont en revanche bien des **flèches pointillées à pointe ouverte**, mais orientées en sens inverse l'une de l'autre :
 - `<<include>>` part du cas de **base** vers le cas **inclus** — consulter le tableau de bord inclut toujours s'authentifier
 - `<<extend>>` part du cas d'**extension** vers le cas de **base** — « créer une réservation » n'est que l'aboutissement optionnel de « converser avec l'assistant », la flèche part donc de la création vers la conversation
- Chaque extension porte sa **condition de déclenchement** entre crochets (`[dialogue abouti]`, `[aucun créneau libre]`)

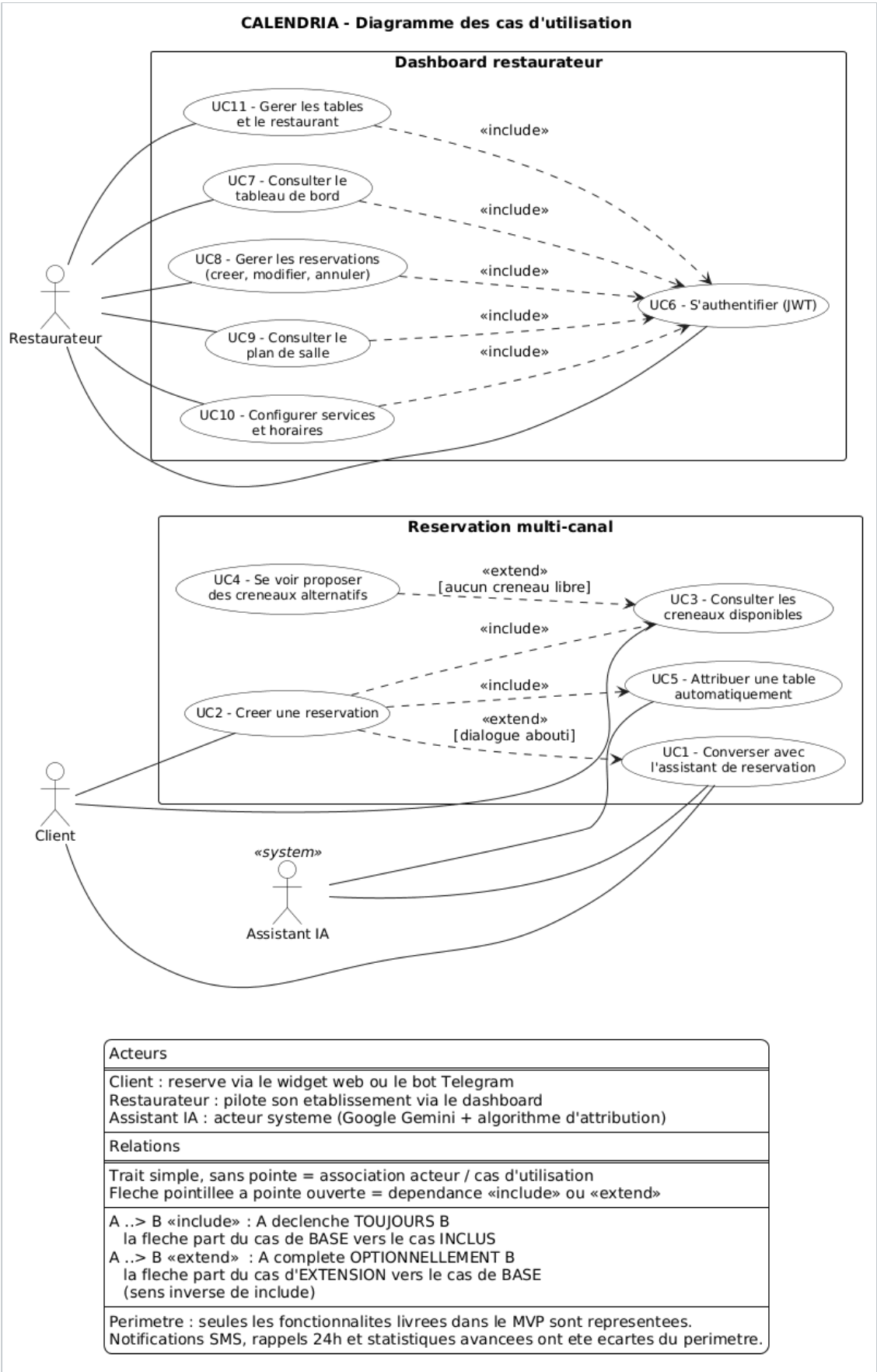


Figure 7.1 - Diagramme des Cas d'Utilisation

2. Vue Dynamique (Séquences)

2.1 Séquence : Réservation via Chatbot

Ce diagramme explicite le cœur d'innovation de CALENDRIA : la création automatique d'une réservation via l'IA. Il montre les itérations de collecte de données entre l'utilisateur, le contrôleur et **Gemini 2.5 Flash**, puis la délégation de la vérification à `DisponibiliteService` avant la persistance via Doctrine.

Deux points de lecture importants :

- **Chaque accès à PostgreSQL est représenté par un couple appel / réponse.** Une écriture sans flèche de retour laisserait croire que l'application n'attend aucune confirmation de la base, ce qui serait faux : le `flush()` Doctrine est synchrone et transactionnel.
- L'état de la conversation est conservé dans le **cache Symfony** et non en base : une session de dialogue est éphémère et n'a pas vocation à être persistée dans le MVP.

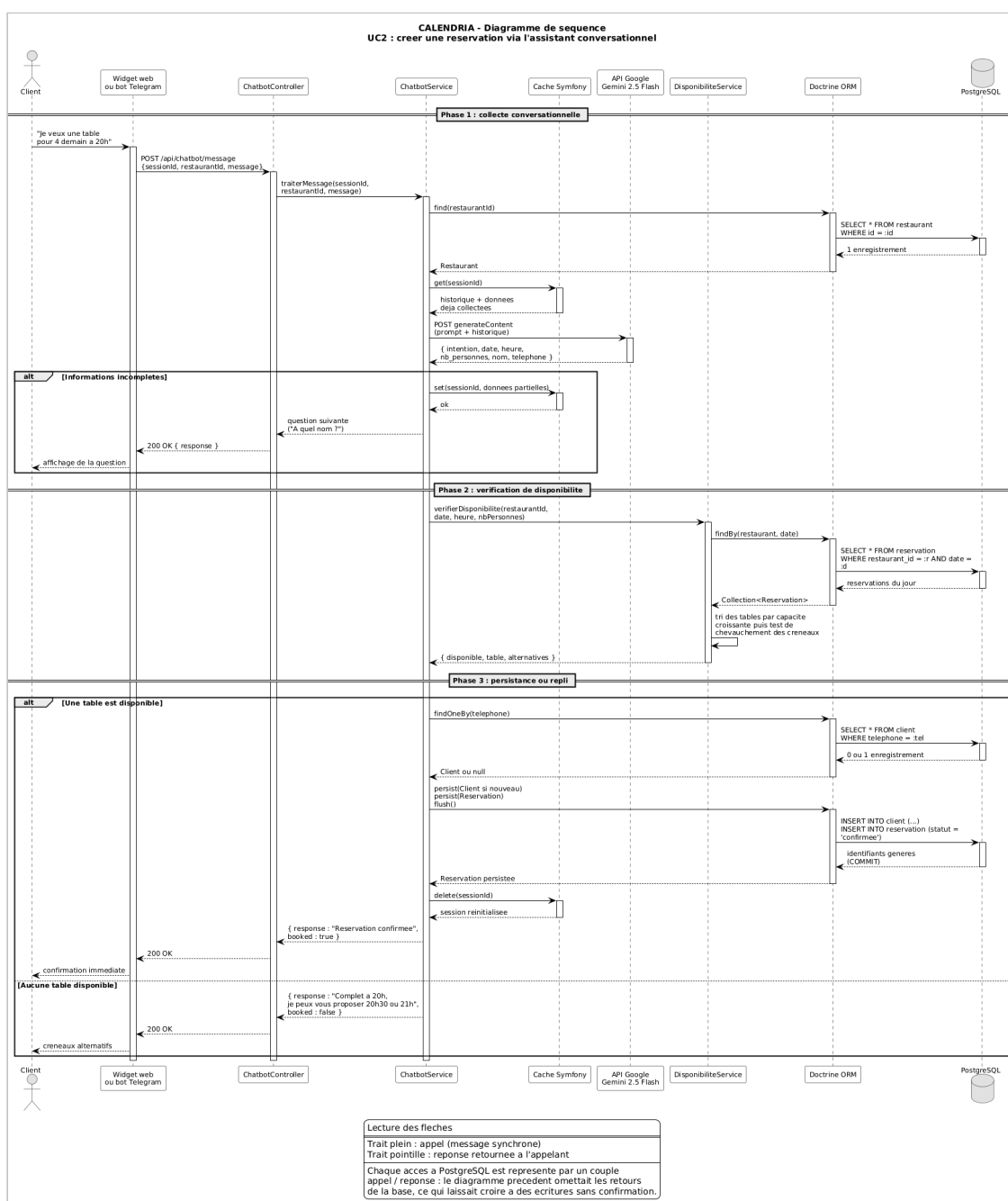


Figure 7.2 - Séquence Réservation via Chatbot IA

2.2 Séquence : Authentification JWT

Documente l'interaction entre le Frontend React (Axios) et le module Symfony LexikJWT pour récupérer un token signé RS256.

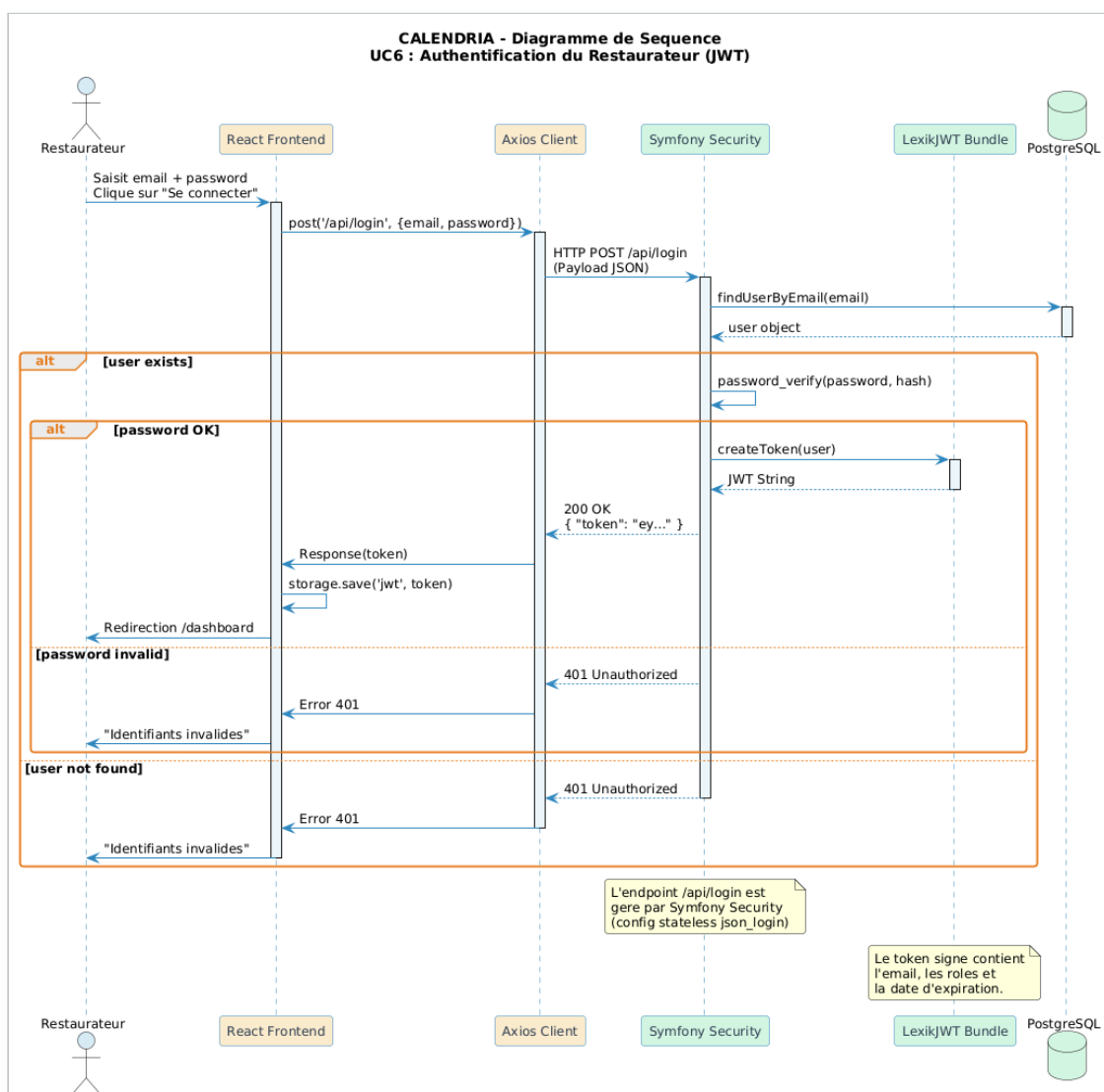


Figure 7.3 - Séquence Authentification JWT

2.3 Séquence : Gestion d'une Réservation

Montre l'utilisation du JWT pour protéger les requêtes et illustre un workflow (ex: Marquer un No-show) provoquant des modifications complexes en base et un effet de bord externe (SMS de notification).

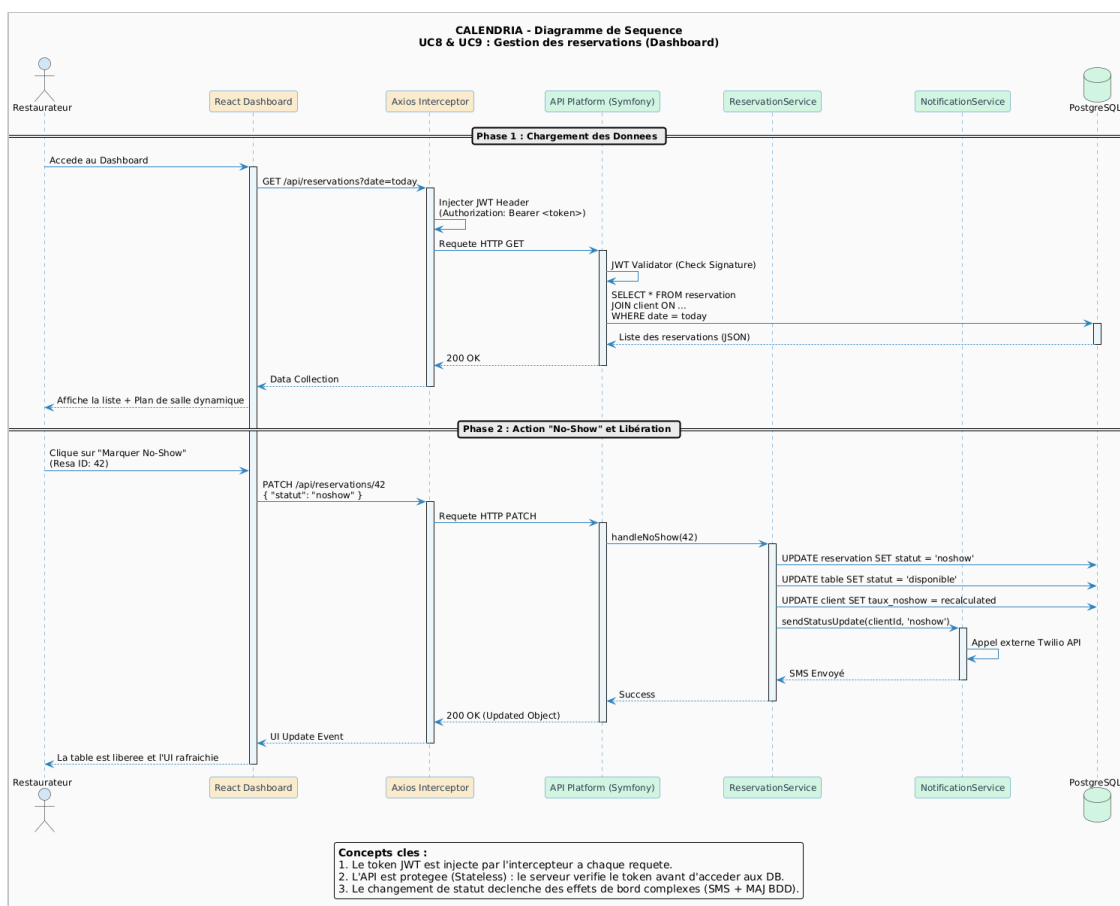


Figure 7.4 - Séquence Gestion Réservation

3. Vue Statique (Classes)

Le diagramme de classes va au-delà du MLD : il montre comment les concepts métier sont traduits en objets PHP et comment les couches communiquent. Il décrit le **code réellement livré** : 6 entités Doctrine, 6 repositories, 2 services métier et 4 contrôleurs, répartis en quatre packages.

- **Entity** - modèles ORM Doctrine, sans logique d'infrastructure
- **Repository** - accès aux données en DQL orienté objet
- **Service** - logique métier complexe, testable hors contexte HTTP
- **Controller** - exposition HTTP de l'API REST

Les évolutions identifiées mais non implémentées dans le MVP sont portées sur le diagramme avec le stéréotype `<<amélioration identifiée>>` : énumérations PHP pour les statuts et les types, attributs dérivés `/tauxNoshow` et `/capaciteTotale`, et association `Restaurant - User` pour le passage en multi-établissement. Ces points sont détaillés en section XV.

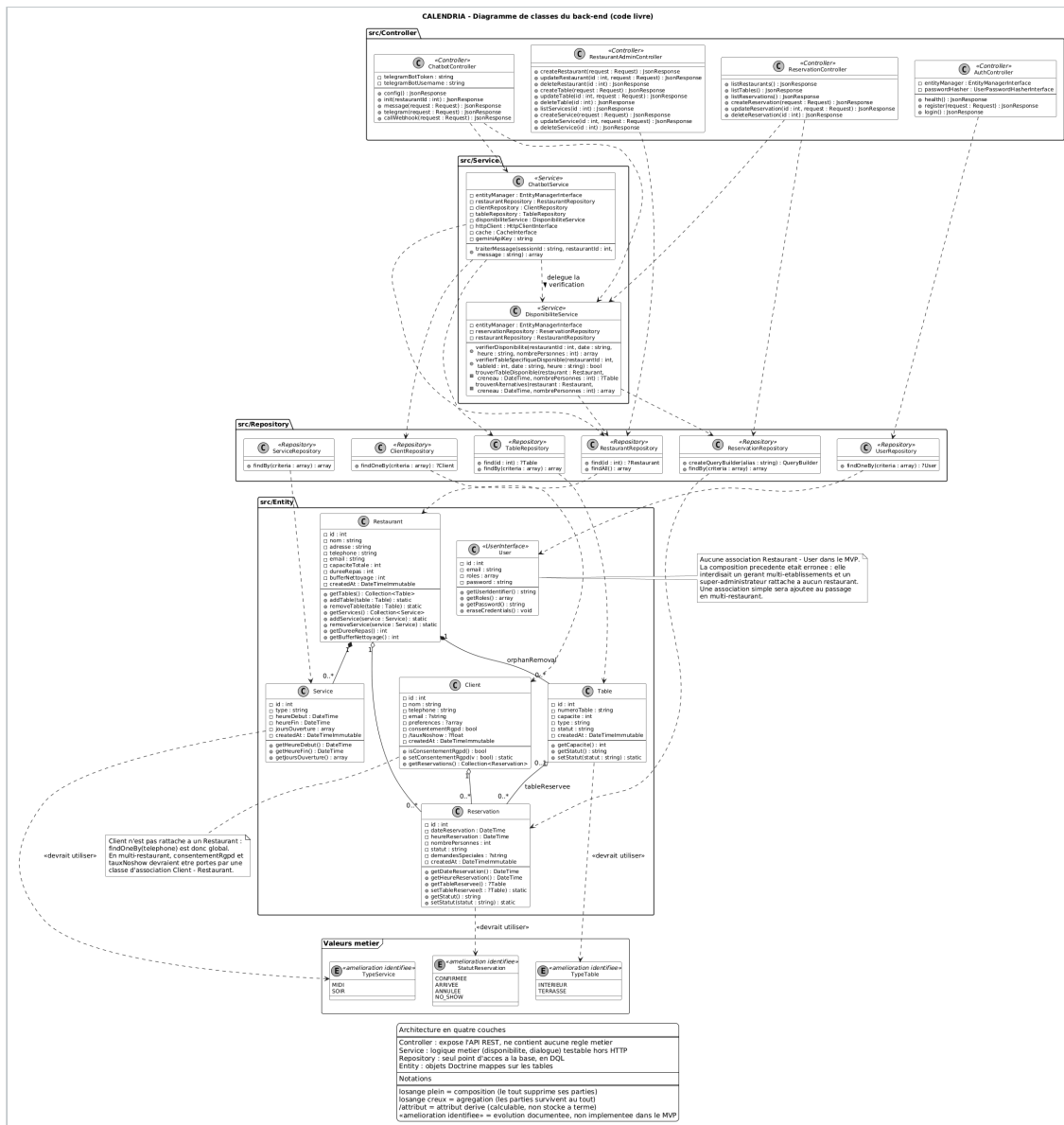


Figure 7.5 - Diagramme de Classes UML

Section VIII - Architecture Multi-Couches

1. Architecture N-Tiers (Déploiement Physique)

L'application suit une **architecture 3-tiers** au sens du déploiement physique :

Clarification 3-tiers vs N-couches : L'architecture **physique** est 3-tiers (navigateur, serveur API, base de données). L'architecture **logique** interne au backend est N-couches (4 couches : Controller, Service, Entity, Repository).

Tier 1 : Présentation (Client)

Navigateur Web / Mobile : Charge la SPA React compilée par Vite. Fichiers statiques servis par nginx. Gère l'affichage (UI), le routage côté client (react-router-dom) et l'UX.

Tier 2 : Logique Applicative (Backend API)

Symfony 6.4 (PHP 8.4) : Reçoit les requêtes HTTP JSON et les webhooks externes. API RESTful
Stateless : authentification via JWT à chaque requête, pas de sessions en mémoire. Configure CORS via NelmioCorsBundle.

Tier 3 : Accès aux Données (SGBD)

PostgreSQL 15 : Base relationnelle protégée du réseau externe, accessible uniquement par le Tier 2 via TCP/IP sur le port interne 5432.

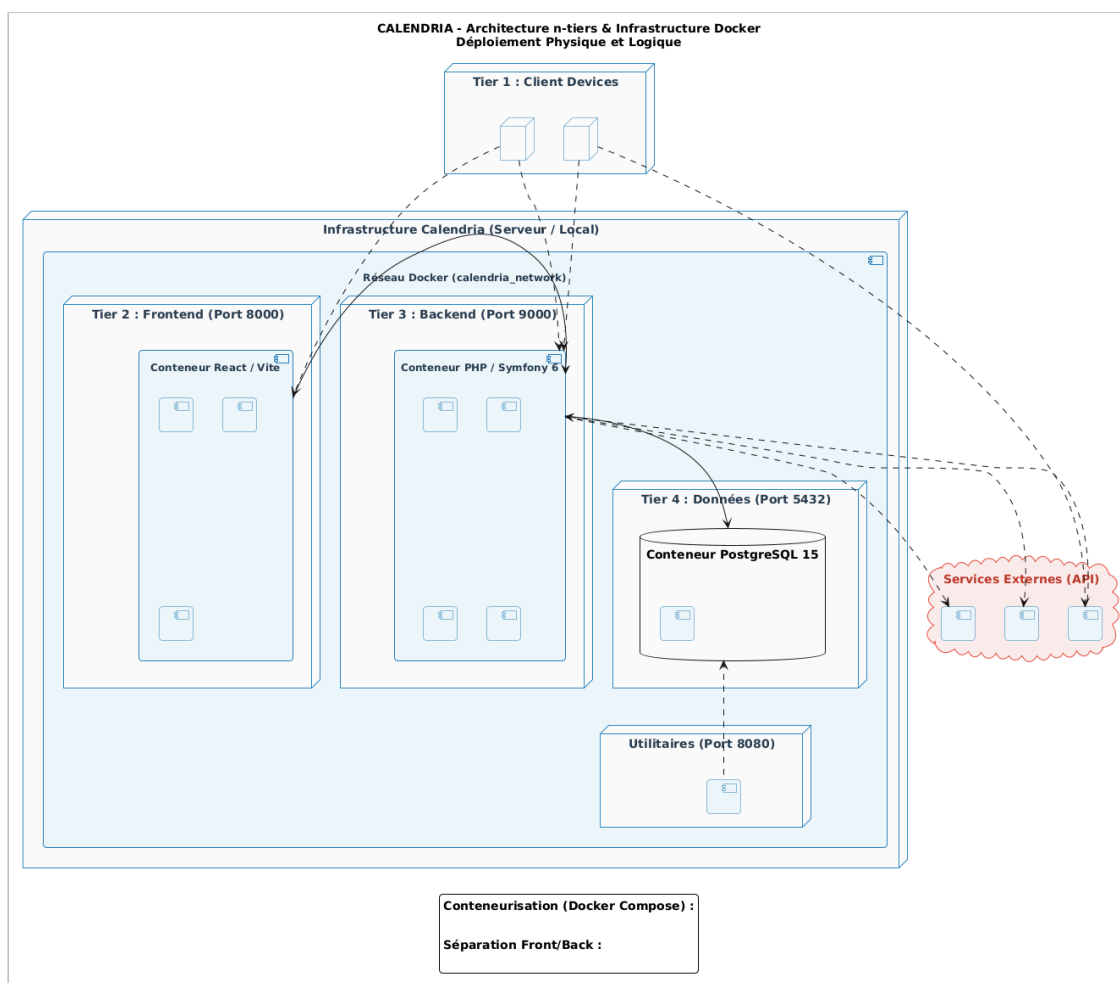


Figure 8.1 - Architecture N-Tiers de CALENDRIA

2. Architecture Logique (Backend Symfony)

Le backend Symfony suit le pattern **MVC étendu vers une architecture hexagonale (orientée services)** :

A. Séparation en 4 Couches

1. **Contrôleurs (Couche HTTP)** : Point d'entrée des requêtes HTTP. Désérialisent les données JSON, appellent les services métier, retournent des réponses HTTP typées (201, 400, 401...). Les endpoints principaux sont des contrôleurs Symfony avec `#[Route]`, offrant un contrôle total.
2. **Services (Couche Métier)** : Logique métier complexe - algorithme de disponibilité et d'attribution des tables (`DisponibiliteService`), dialogue avec l'IA et orchestration de la réservation (`ChatbotService`). Indépendants du contexte HTTP, donc testables unitairement.
3. **Entités (Couche Modèle)** : POPO (Plain Old PHP Objects) représentant les entités métier. Pas de logique d'infrastructure, uniquement propriétés et getters/setters.
4. **Repositories (Couche Persistance)** : Data Access Layer via Doctrine ORM. Les requêtes SQL complexes (ex: réservations du jour) résident dans les repositories, pas dans les contrôleurs.

B. Principes SOLID Appliqués

- **SRP** : La séparation Controller / Service / Repository assure qu'une classe n'a qu'une seule raison de changer

- **OCP** : L'ajout des canaux Telegram et vocal n'a demandé aucune modification de `DisponibiliteService` : chaque canal est un nouveau point d'entrée qui réutilise le même service métier
 - **DIP** : L'Injection de Dépendances Symfony - les contrôleurs reçoivent les services dans leur constructeur
-

3. Composants Externes et Bibliothèques

Back-end (Bundles Symfony)

- **API Platform** : Installé mais non exploité dans le MVP — les 24 endpoints sont des contrôleurs Symfony écrits à la main pour garder un contrôle total sur la validation et les codes de retour. Son exploitation (documentation OpenAPI automatique) est listée en perspectives.
- **Lexik JWT Authentication** : Création et validation des tokens JWT RS256
- **Nelmio CORS** : Politique CORS-Origin pour appels depuis le domaine frontend
- **Doctrine ORM** : Pont entre les objets PHP et PostgreSQL, génération des migrations SQL

APIs Tierces

- **Google Gemini 2.5 Flash** : NLP du chatbot - interprète les intentions et extrait les variables (date, heure, couverts)
- **Retell AI** : Agent vocal IA - gère les appels téléphoniques, appelle le webhook `/api/chatbot/call`
- **Telegram Bot API** : Canal de messagerie - webhook `/api/chatbot/telegram`

Section IX - Sécurité et Conformité

1. Politique de Sécurité Applicative (OWASP Top 10)

L'architecture de CALENDRIA a été conçue pour respecter rigoureusement les préconisations de sécurité de l'OWASP.

a) Injection SQL (A03:2021)

- **Protection native via l'ORM** : Doctrine ORM utilise des requêtes préparées systématiques, séparant hermétiquement la logique SQL des données utilisateur
- **QueryBuilder** : Toutes les variables externes (`$restaurantId`, `$dateStr` ...) injectées via `->setParameter()` . Aucune concaténation brute de chaîne tolérée

b) Failles XSS (A03:2021)

- **Échappement automatique React** : Toutes les variables dans le JSX sont échappées par défaut, prévenant l'exécution de scripts malveillants
- **Échappement Twig** : Pour les pages backend (erreurs, profil), échappement HTML par défaut sur toutes les variables

c) Attaques CSRF (A01:2021)

- **API Stateless avec JWT** : Les requêtes du frontend n'utilisent pas de sessions/cookies de navigation, rendant l'application insensible au CSRF
- Le token JWT est stocké en mémoire React et envoyé dans `Authorization: Bearer <JWT_TOKEN>`

d) Brute Force (A07:2021)

- **Rate Limiting** : `LoginRateLimiterSubscriber` - 5 tentatives max / 15 min par IP
- Blocage IP automatique via cache Symfony après dépassement

e) Stockage des Mots de Passe (A02:2021)

- **Argon2id** : Algorithme de hachage par défaut de Symfony 6.4 - aucun mot de passe stocké en clair
- Politique de mot de passe robuste exigée à la création de compte

f) Contrôle d'Accès & Firewalls (A01:2021)

- **Firewall Principal (/api)** : Protège tous les endpoints d'administration - JWT valide requis (401 sinon)
- **Firewall Public (/api/chatbot)** : Accès public autorisé pour le Widget Web, le bot Telegram et les webhooks - limité aux actions conversationnelles

2. Conformité RGPD

CALENDRIA manipule des données à caractère personnel (nom, téléphone, email). Les mesures suivantes garantissent la conformité :

Exigence RGPD	Implémentation dans CALENDRIA
Minimisation des données	Collecte uniquement : Nom, Téléphone, Heure, Couverts. L'email est optionnel
Consentement Explicite	Champ <code>consentement_rgpd</code> (BOOLEAN) porté par l'entité Client, positionné à la création de la réservation quel que soit le canal
Sécurisation des flux	HTTPS obligatoire en production (Let's Encrypt via Railway)
Droit à l'Oubli	Suppression des fiches clients depuis le dashboard - cascade en base de données
Durée de conservation	Données clients : 2 ans après dernière réservation - Logs : 1 an maximum

Note : CALENDRIA manipule des **données personnelles simples** (pas de données de santé sensibles), donc RGPD simplifié par rapport au secteur médical.

Section X - Politique de Tests

1. Stratégie de Tests - Vue d'Ensemble

La qualité logicielle de CALENDRIA repose sur trois niveaux de validation complémentaires :

Niveau	Outil	Périmètre	Déclenchement
Tests unitaires	PHPUnit	Services métier isolés (mocks)	Manuel + CI
Tests fonctionnels	PHPUnit + KernelBrowser	Endpoints HTTP bout-en-bout	Manuel + CI
Type-checking	TypeScript (<code>tsc -b</code>)	Contrats d'interfaces frontend	CI (npm build)
Validation manuelle	Navigateur + Postman	UX, chatbot, plan de salle	À chaque sprint

2. Tests Backend (PHPUnit)

2.1 Organisation des Tests

```

backend/tests/
├─ bootstrap.php
├─ Controller/
│   └─ AuthControllerTest.php      # 6 tests
│   └─ ChatbotControllerTest.php  # 2 tests
├─ Service/
│   └─ DisponibiliteServiceTest.php # 4 tests
└─

Total : 12 tests, 0 failure, 0 error

```

2.2 DisponibiliteServiceTest - Tests Unitaires (4 tests)

Méthode de test	Scénario	Assertion principale
<code>testVerifierDisponibiliteRestaurantNonTrouve</code>	ID restaurant inexistant (999)	<code>disponible === false</code> , <code>table === null</code>
<code>testVerifierDisponibiliteRetourneRaisonFerme</code>	Aucun service configuré	<code>disponible === false</code> , <code>raison === 'ferme'</code>
<code>testVerifierDisponibiliteAvecServiceOuvert</code>	Service ouvert, tables disponibles	<code>disponible === true</code> , <code>table non null</code>
<code>testVerifierDisponibiliteDateInvalide</code>	Format de date invalide	<code>disponible === false</code> , <code>alternatives vide</code>

2.3 AuthControllerTest - Tests Fonctionnels (6 tests)

Méthode de test	Endpoint	Scénario	Statut attendu
<code>testHealthEndpointReturnsOk</code>	GET <code>/api/health</code>	Health check	200
<code>testRegisterWithValidData</code>	POST <code>/api/register</code>	Email + password valides	201
<code>testRegisterWithMissingEmail</code>	POST <code>/api/register</code>	Payload sans email	400
<code>testRegisterWithMissingPassword</code>	POST <code>/api/register</code>	Payload sans password	400
<code>testRegisterWithDuplicateEmail</code>	POST <code>/api/register</code>	Email déjà utilisé	409
<code>testLoginWithInvalidCredentials</code>	POST <code>/api/login</code>	Mauvais mot de passe	401

2.4 ChatbotControllerTest - Tests Fonctionnels (2 tests)

Méthode de test	Scénario	Statut attendu
<code>testCallEndpointRequiresRestaurantId</code>	Payload sans <code>restaurantId</code>	400
<code>testCallEndpointRetourneMessageErreurSiRestaurantInexistant</code>	<code>restaurantId: 99999</code>	200 (message d'erreur Retell)

Choix d'architecture : L'endpoint `/api/chatbot/call` retourne toujours 200 OK - Retell AI interprète tout code non-200 comme erreur fatale. Les erreurs métier sont communiquées dans le corps JSON (`booking_message`).

3. Tests Frontend

3.1 TypeScript Strict Mode

Le frontend React est configuré en mode TypeScript **strict** (`tsconfig.app.json`) :

```
{
  "compilerOptions": {
    "strict": true,
    "noImplicitAny": true,
    "strictNullChecks": true
  }
}
```

`npm run build` exécute `tsc -b` avant Vite, rejetant toute erreur de type. C'est le filet de sécurité pour les interfaces de services.

3.2 Interfaces TypeScript Validées

```
interface Reservation { id: number; date: string; heure: string; ... }
interface ServiceItem { id: number; typeService: string; heureOuverture: string;
... }
interface RestaurantData { id: number; nom: string; tables: Table[]; services:
ServiceItem[]; ... }
```

4. Couverture des Cas Critiques

Cas Limite	Test Couvrant
Restaurant inexistant	testVerifierDisponibiliteRestaurantNonTrouve
Aucun service ouvert	testVerifierDisponibiliteRetourneRaisonFerme
Date au format invalide	testVerifierDisponibiliteDateInvalide
Inscription email dupliqué	testRegisterWithDuplicateEmail
Login mauvaises credentials	testLoginWithInvalidCredentials
Webhook sans restaurantId	testCallEndpointRequiresRestaurantId

5. Résultats CI - État Final

```
PHPUnit 11.x - CALENDRIA Test Suite
.....

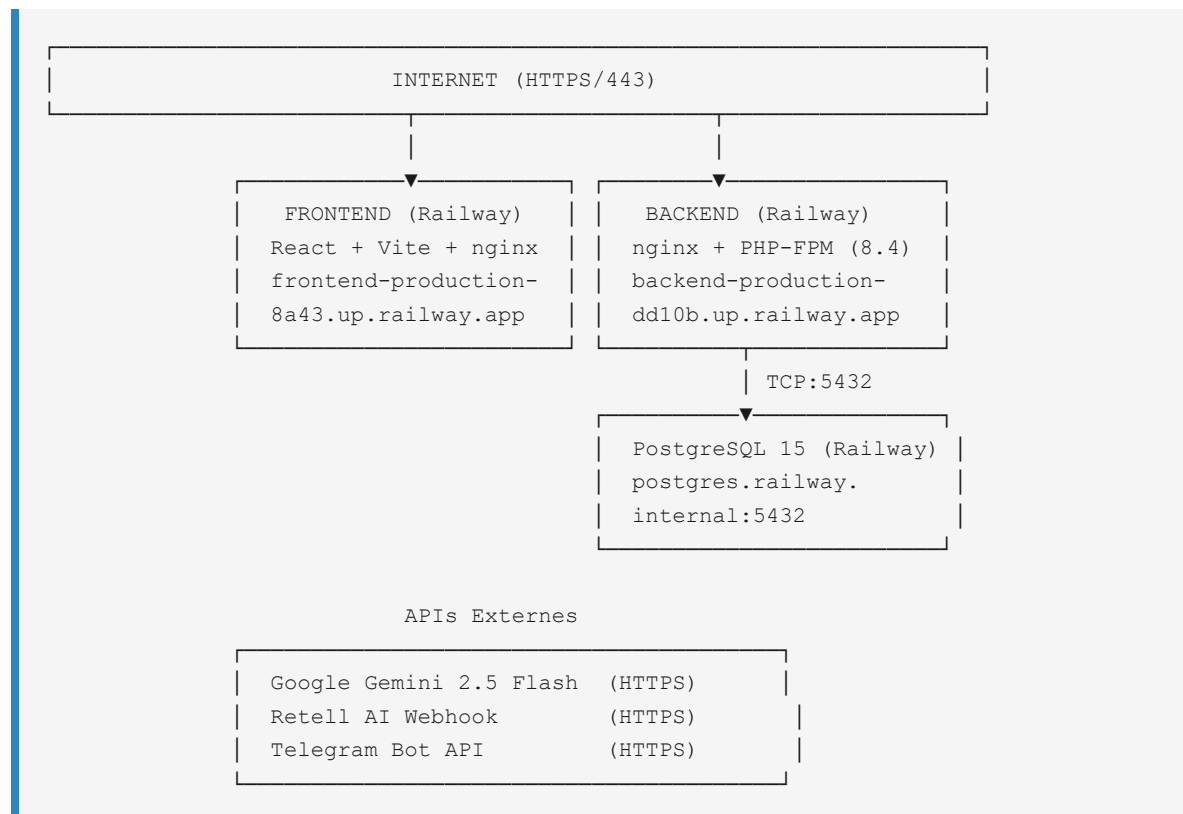
OK (12 tests, 24 assertions)
Time: 00:02.841, Memory: 32.00 MB
```

Badge CI :  CI/CD Pipeline failing

Section XI - Déploiement et Mise en Production

1. Architecture de Production

1.1 Vue d'Ensemble - Railway



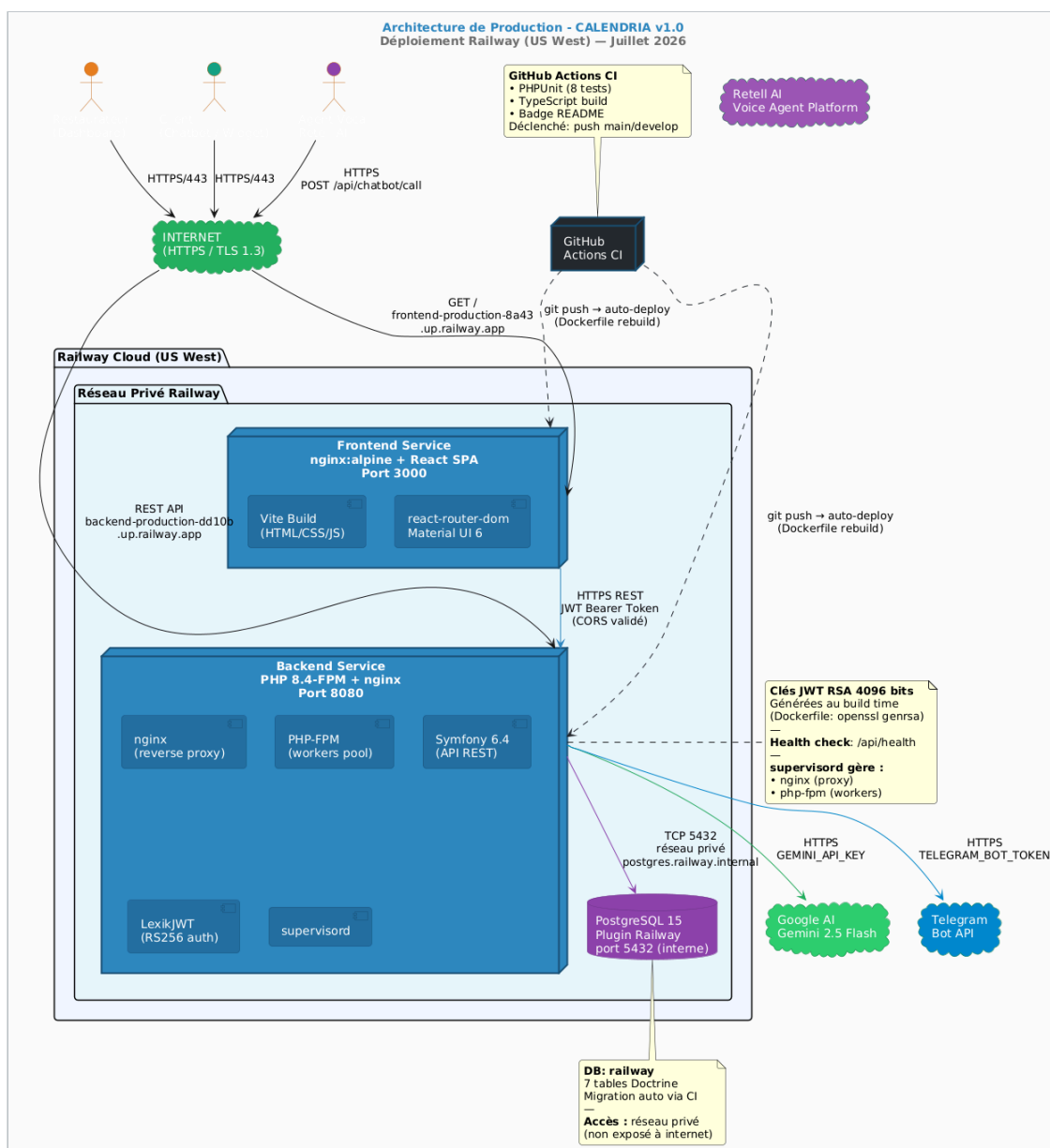


Figure 11.1 - Architecture de Production Railway

1.2 Sécurité en Production

- **TLS/HTTPS** : Terminaison SSL par Railway (Let's Encrypt automatique)
- **CORS** : NelmioCorsBundle filtre par regex (`^https://frontend-production-8a43\.up\.railway\.app$`)
- **JWT RSA** : Clés générées au build time (`openssl genrsa 4096`), stockées dans l'image Docker
- **Rate Limiting** : 5 tentatives de login échouées → blocage IP 15 minutes

2. Containerisation Docker

2.1 Backend - Build Multi-Stage

```

Étape 1 : Image de base (php:8.4-fpm-alpine)
  ↓ Installation des extensions PHP (pdo_pgsql, opcache, intl...)
  ↓ Installation de nginx + supervisord
  ↓ composer install --no-dev
  ↓ openssl genrsa 4096 (clés JWT)
  ↓ cache:warmup (mode prod)
  ↓ EXPOSE 8080

```

```

Entrypoint Runtime (entrypoint.sh) :
  ↓ Lecture de $PORT (fallback 8080)
  ↓ Écriture dynamique config nginx (sed __PORT__)
  ↓ php bin/console cache:clear + cache:warmup
  ↓ exec /usr/bin/supervisord (nginx + php-fpm)

```

2.2 Frontend - Multi-Stage

```

Étape 1 : Build (node:20-alpine)
  ↓ npm install
  ↓ VITE_API_URL=${ARG} npm run build
  → /app/dist (assets statiques)

Étape 2 : Serve (nginx:alpine)
  ↓ COPY --from=build /app/dist /usr/share/nginx/html
  ↓ Config SPA fallback (try_files $uri /index.html)
  ↓ EXPOSE 3000

```

Point clé : `VITE_API_URL` est injecté au **build time** - l'image doit être reconstruite si l'URL backend change.

2.3 Docker Compose (Développement Local)

```

# Démarrage complet (première fois)
docker compose up -d --build

# Migrations
docker compose exec backend php bin/console doctrine:migrations:migrate --no-
interaction

# Données de démo
docker compose exec backend php bin/console doctrine:fixtures:load --no-interaction

# Tests
docker compose exec backend vendor/bin/phpunit

```

3. Pipeline CI/CD (GitHub Actions)

Fichier `.github/workflows/ci.yml` - déclenchée à chaque push/PR sur `main`, `master` et `develop`.

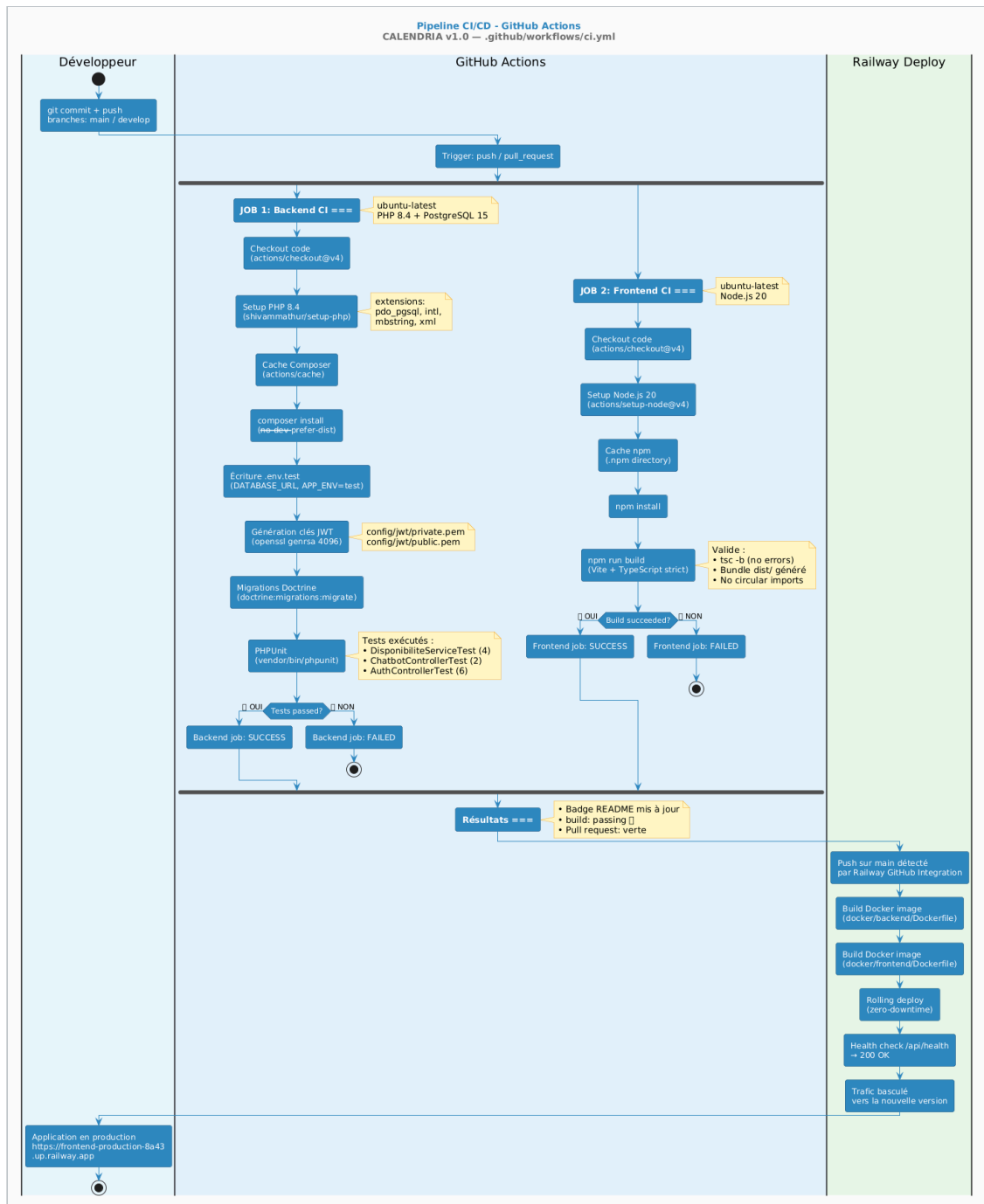


Figure 11.2 - Pipeline CI/CD GitHub Actions

4. Variables d'Environnement (Production)

Variable	Description
APP_ENV	prod
APP_SECRET	Clé secrète Symfony (32 caractères)
DATABASE_URL	URL complète PostgreSQL interne Railway
CORS_ALLOW_ORIGIN	Regex URL frontend

Variable	Description
GEMINI_API_KEY	Clé API Google Gemini 2.5
TELEGRAM_BOT_TOKEN	Token du bot Telegram
JWT_PASSPHRASE	Passphrase des clés RSA

Principe : Aucun secret dans le code source ni dans le dépôt Git. Variables gérées via le dashboard Railway (chiffrées au repos).

5. Instructions de Démarrage Local

```
# Étape 1 : Cloner le dépôt
git clone https://github.com/SafiBougherara/projet_fin_annee.git
cd projet_fin_annee

# Étape 2 : Configurer les variables d'environnement
cp backend/.env.example backend/.env
# Renseigner GEMINI_API_KEY dans backend/.env

# Étape 3 : Construire et démarrer tous les services
docker compose up -d --build

# Étape 4 : Initialiser la base de données
docker compose exec backend php bin/console doctrine:migrations:migrate --no-
interaction
docker compose exec backend php bin/console doctrine:fixtures:load --no-interaction

# Étape 5 : Accéder à l'application
# Dashboard restaurateur → http://localhost:3000
# API REST (documentation) → http://localhost:8000/api
# Health check → http://localhost:8000/api/health

# Identifiants de démo
# Email : admin@calendria.com
# Password : password123
```

6. Guide du Restaurateur

6.1 Configuration Initiale

Étape 1 - Créer votre restaurant : Menu → Gestion des Restaurants → Ajouter. Renseigner nom, adresse, téléphone, durée d'un repas et marge de nettoyage.

Étape 2 - Créer les tables : Depuis la fiche restaurant → Ajouter une Table → numéro, capacité, type.

Étape 3 - Définir les services : Section Horaires → Ajouter un service → type (Déjeuner/Dîner), heures, jours.

⚠ Sans service configuré, le chatbot ne peut pas prendre de réservations

6.2 Canaux de Réservation

Widget Web : URL `/widget?restaurantId=X` - intégrable via iframe sur le site du restaurant.

Telegram : Configurez le webhook : `POST https://api.telegram.org/bot{TOKEN}/setWebhook?url=https://backend.../api/chatbot/telegram?restaurantId=X`

Retell AI (Vocal) : Webhook public `POST /api/chatbot/call` - paramètres : `name, phone, date, time, guests, restaurantId`

Section XII - Veille Technologique

1. Définition et Démarche

La veille technologique consiste à surveiller activement l'évolution des technologies utilisées ou susceptibles d'impacter le projet, afin d'anticiper les changements et d'adopter les meilleures pratiques. Dans le cadre de CALENDRIA, cette veille a été continue tout au long des 6 mois de développement et a directement influencé plusieurs choix techniques.

2. Technologies Surveillées

2.1 Intelligence Artificielle Générative

Le domaine de l'IA générative est en évolution rapide. Pour CALENDRIA, la sélection de l'IA conversationnelle a été un choix stratégique révisé en cours de projet :

Solution	Évaluée	Statut	Raison
OpenAI GPT-4o-mini	✓ Oui	Remplacée	Coûts en production, limite gratuite insuffisante
Google Gemini 1.5 Flash	✓ Oui	Évaluée	Bonnes performances mais tier gratuit limité
Google Gemini 2.5 Flash	✓ Oui	Retenue	Tier gratuit généreux, excellentes performances NLP, API stable
Mistral AI (open source)	✓ Oui	Non retenue	Qualité NLP inférieure pour le français conversationnel
Anthropic Claude	✓ Oui	Non retenue	Pas de tier gratuit exploitable pour le développement

Évolution notable : Gemini 2.5 Flash intègre désormais un mode "thinking" permettant un raisonnement en plusieurs étapes avant de répondre. Cette capacité améliore la compréhension des demandes ambiguës (ex: "pour ce soir" → inférence de la date du jour).

2.2 Écosystème PHP / Symfony

Technologie	Version Utilisée	Tendances 2026
PHP	8.4	PHP 8.4 introduit les property hooks et l'asymmetric visibility - améliorations ergonomiques majeures
Symfony	6.4 LTS	Symfony 7.x en développement actif - support LTS 6.4 jusqu'en 2027
Doctrine ORM	3.x	Support natif des nouvelles fonctionnalités PHP 8.x
API Platform	3.x	Migration vers State Providers/Persistsers (successeurs des DataProviders v2)

2.3 Écosystème JavaScript / React

Technologie	Version Utilisée	Tendances 2026
React	18	React 19 introduit les Server Components stables et les nouvelles hooks (use, useOptimistic)
TypeScript	5.x	TypeScript 5.5+ améliore l'inférence de types complexes
Vite	5.x	Vite 6 améliore les temps de build en environnement CI

2.4 Infrastructure et DevOps

Technologie	Utilisation	Alternatives Évaluées
Docker / Docker Compose	Containerisation complète	Podman (compatible, mais moins d'intégrations)
Railway	Hébergement production	Heroku (plus cher), Fly.io (bonne alternative), Render (gratuit mais lent)
GitHub Actions	CI/CD	GitLab CI (si hébergement GitLab), CircleCI (plus complexe)

2.5 Canaux de Communication - Évolution du Choix

La veille sur les APIs de messagerie a conduit à des révisions importantes :

Canal	Évalué	Décision	Justification
WhatsApp Business (Meta)	✓ Oui	✗ Abandonné	Vérification Business Account requise, délais de 4-6 semaines, coûts API
Twilio SMS/WhatsApp	✓ Oui	✗ Abandonné	Sandbox limité, coûts dès la production
Telegram Bot API	✓ Oui	✓ Retenu	Totalement gratuit, webhook instantané, API stable et bien documentée
Retell AI (Vocal)	✓ Oui	✓ Retenu (Bonus)	Tier gratuit disponible, démonstration vocale impressionnante

3. Outils de Veille Utilisés

- **Documentation officielle** : Symfony Docs, React Docs, Google AI Docs - consultées hebdomadairement
- **GitHub Release Notes** : Suivi des changelogs des dépendances via Dependabot
- **Developer Blogs** : Symfony Blog, React Blog, PHP.Watch
- **Communautés** : Stack Overflow, Reddit (r/PHP, r/symfony, r/reactjs)
- **Conférences en ligne** : SymfonyWorld, ReactConf (replays YouTube)

4. Impact sur le Projet

La veille technologique a directement influencé 3 décisions majeures :

1. **Passage de OpenAI GPT-4o-mini → Google Gemini 2.5 Flash** : Économie des coûts API, tier gratuit suffisant pour le MVP
2. **Passage de Twilio WhatsApp → Telegram Bot** : Suppression des barrières administratives, déploiement en quelques minutes
3. **Choix de Railway plutôt qu'Heroku** : Heroku a supprimé son tier gratuit en 2022 - Railway offre une alternative moderne avec support Docker natif

Section XIII - Veille Sécurité

1. Définition et Importance

La veille sécurité consiste à surveiller les nouvelles vulnérabilités, attaques et contre-mesures pour garantir que l'application reste protégée tout au long de sa durée de vie. Dans le secteur de la restauration, bien que les données ne soient pas aussi sensibles que dans la santé ou la finance, la protection des données personnelles (nom, téléphone) reste une obligation légale (RGPD) et éthique.

2. Sources de Veille Sécurité Consultées

Source	Contenu	Fréquence
OWASP Top 10	Référentiel des 10 risques les plus critiques pour les applications web	Mensuelle
Symfony Security Advisories	Bulletins de sécurité Symfony - correctifs à appliquer	À chaque release
GitHub Security Advisories	CVE sur les dépendances Composer et npm	Automatique (Dependabot)
CNIL - Sécurité	Recommandations CNIL sur la protection des données	Trimestrielle

3. Menaces Spécifiques à CALENDRIA

3.1 Risques liés aux APIs Externes

CALENDRIA s'appuie sur 3 APIs externes (Gemini, Telegram, Retell AI). Chaque intégration crée un vecteur d'attaque potentiel :

Menace	Vecteur	Contre-mesure Implémentée
Fuite de clé API	Clé committée dans Git	Variables d'environnement Railway + <code>.gitignore</code> sur <code>.env</code>
Webhook spoofing	Requête falsifiée sur <code>/api/chatbot/call</code>	Firewall public limité aux actions conversationnelles sans accès aux données restaurateur
Déni de service via chatbot	Spam de messages sur le bot Telegram	Rate limiting Symfony + quotas API Gemini côté Google

3.2 Risques liés à l'Authentification

Menace	Contre-mesure
Brute force sur <code>/api/login</code>	<code>LoginRateLimiterSubscriber</code> : 5 tentatives / 15 min par IP
Vol de token JWT	Stockage en mémoire React (pas dans localStorage/cookies), expiration courte
Token JWT non révocable	Rotation automatique à chaque redéploiement (nouvelles clés RSA)

Menace	Contre-mesure
Élévation de privilèges	Firewall Symfony distinguant <code>/api</code> (JWT requis) et <code>/api/chatbot</code> (public)

3.3 Risques liés aux Données Personnelles

Menace	Contre-mesure
Exfiltration en base de données	PostgreSQL accessible uniquement sur réseau privé Railway (pas d'exposition publique)
Injection via formulaires	Validation Symfony Validator sur tous les inputs, ORM Doctrine pour les requêtes
Interception des données en transit	HTTPS obligatoire (Let's Encrypt) sur les deux domaines Railway
Violation RGPD	Consentement explicite, droit à l'oubli, données minimales collectées

4. Veille sur les Dépendances (Supply Chain Security)

Les attaques sur la chaîne d'approvisionnement logicielle (supply chain attacks) sont en forte hausse depuis 2021 (ex: incident SolarWinds, event-stream npm). Pour CALENDRIA :

- **Composer** : `composer audit` exécuté régulièrement pour détecter les CVE sur les packages PHP
- **npm** : `npm audit` intégré au pipeline CI/CD
- **Dependabot** (GitHub) : Alertes automatiques sur les vulnérabilités connues dans les dépendances
- **Versions LTS** : Choix délibéré de Symfony 6.4 LTS (support sécurité jusqu'en 2027) pour minimiser les mises à jour forcées

5. Incidents Sécurité Traités en Cours de Projet

Incident 1 - Exposition des clés JWT

Situation : Les clés JWT RSA étaient initialement générées localement et risquaient d'être committées par erreur. **Solution** : Génération des clés directement dans le `Dockerfile` au build time - les clés ne transitent jamais dans le dépôt Git. Ajout d'un `.gitignore` couvrant `config/jwt/*.pem`.

Incident 2 - Headers d'information exposés

Situation : Le header `X-Powered-By: PHP/8.4` exposait la version du serveur dans les réponses API. **Solution** : Suppression dans la configuration nginx (`fastcgi_hide_header X-Powered-By;`) et ajout de `expose_php = Off` dans `php.ini`.

6. Perspectives Sécurité

Si le projet était poursuivi, les améliorations sécurité prioritaires seraient :

1. **Content Security Policy (CSP)** : En-tête HTTP définissant les sources de contenu autorisées - protection XSS avancée
2. **Audit de sécurité externe** : Scan OWASP ZAP ou Burp Suite sur l'API en production

3. **SIEM** : Centralisation des logs de sécurité (Sentry ou Elastic Stack) pour détecter les tentatives d'intrusion
4. **Rotation automatique des clés JWT** : Mécanisme de refresh token pour éviter l'invalidation à chaque déploiement

Section XIV - Difficultés Rencontrées et Solutions

Le développement de CALENDRIA a confronté plusieurs obstacles techniques significatifs. Chaque problème a été une opportunité d'apprentissage et a conduit à une solution documentée.

Problème 1 - Configuration CORS en Production

Symptôme : Les requêtes du frontend vers le backend échouaient avec l'erreur `CORS policy: No 'Access-Control-Allow-Origin'` en production.

Cause : L'URL du backend Railway (auto-générée) ne correspondait pas à la configuration CORS statique de Symfony. De plus, Railway peut modifier le sous-domaine lors de redéploiements.

Solution : Variable d'environnement `CORS_ALLOW_ORIGIN` configurée comme regex dans Railway (`^https://frontend-production-8a43\.up\.railway\.app$`). NelmioCorsBundle lit cette variable à l'exécution, permettant une configuration dynamique sans rebuild.

Problème 2 - Clés JWT Perdues entre Redéploiements

Symptôme : Tous les utilisateurs étaient déconnectés à chaque redéploiement de l'application en production.

Cause : Les clés JWT RSA sont générées au build time dans le Dockerfile (`openssl genrsa 4096`). Chaque rebuild génère de nouvelles clés, invalidant les tokens existants.

Solution : Comportement documenté et accepté pour le MVP (déploiements rares, usage professionnel). La solution long terme serait de stocker les clés RSA dans un secret Railway et de les monter au démarrage plutôt que de les générer.

Problème 3 - Variables d'Environnement Telegram/Gemini Non Définies

Symptôme : Le conteneur Docker backend plantait au démarrage avec `Environment variable "TELEGRAM_BOT_TOKEN" is not defined`, même quand la fonctionnalité Telegram n'était pas utilisée.

Cause : `services.yaml` injectait ces variables dans tous les services via la section `bind` de Symfony, rendant leur présence obligatoire au démarrage du container, même si les services associés ne sont pas activés.

Solution : Déclaration de valeurs par défaut vides dans la section `parameters` de `services.yaml` :

```
parameters:
    telegram_bot_token: '%env(default::TELEGRAM_BOT_TOKEN)%'
    gemini_api_key: '%env(default::GEMINI_API_KEY)%'
```

Problème 4 - Doublement du Suffixe `_test` dans la CI

Symptôme : La CI échouait avec `FATAL: database "calendria_test_test" does not exist`

Cause : `DATABASE_URL` contenait déjà `calendria_test` dans le workflow GitHub Actions. Symfony ajoute automatiquement le suffixe `_test` quand `APP_ENV=test`, ce qui donnait `calendria_test_test`.

Solution : `DATABASE_URL` pointe vers la base `calendria` (sans suffixe) - Symfony ajoute `_test` → connexion à `calendria_test`. Le service PostgreSQL du CI crée donc la base `calendria_test` directement.

Problème 5 - Port Mismatch sur Railway

Symptôme : L'application backend répondait en local mais retournait `502 Bad Gateway` sur Railway.

Cause : Railway utilise une variable d'environnement dynamique `$PORT` pour exposer le conteneur sur un port aléatoire. La configuration nginx du Dockerfile était hardcodée sur le port 8080, mais Railway attendait que le conteneur écoute sur `$PORT`.

Solution : Le script `entrypoint.sh` lit la variable `$PORT` au démarrage et réécrit dynamiquement la configuration nginx via `sed` avant de lancer supervisord :

```
PORT=${PORT:-8080}
sed -i "s/__PORT__/${PORT}/g" /etc/nginx/conf.d/default.conf
exec /usr/bin/supervisord -c /etc/supervisord.conf
```

Section XV - Conclusion et Perspectives

1. Bilan Fonctionnel Final

1.1 Fonctionnalités Livrées vs Planifiées

Fonctionnalité Prévue (CDCF)	Statut	Notes
Dashboard restaurateur avec CRUD réservations	✓ Livré	Complet : liste, détail, création, modification, annulation
Plan de salle interactif avec statuts en temps réel	✓ Livré	Slider temporel, tooltips, codes couleur
Gestion des restaurants et des tables	✓ Livré	CRUD complet avec protection d'intégrité
Authentification sécurisée (JWT)	✓ Livré	RS256 via LexikJWTAuthenticationBundle
Chatbot IA (canal Web Widget)	✓ Livré	Gemini 2.5 Flash, extraction NLP, réservation automatique
Canal Telegram Bot	✓ Livré	Webhook, gestion de session, confirmation
Agent Vocal IA (Retell AI)	✓ Livré	Webhook public, messages d'erreur intelligents
Algorithme de disponibilité intelligente	✓ Livré	Attribution table capacité min., alternatives horaires
Mode sombre (Dark Mode)	✓ Livré	Persistant en localStorage
Containerisation Docker	✓ Livré	nginx + PHP-FPM + supervisord, build multi-stage
Déploiement en production	✓ Livré	Railway (backend + frontend + PostgreSQL)
Pipeline CI/CD	✓ Livré	GitHub Actions : PHPUnit + TypeScript à chaque push
Protection brute-force login	✓ Livré	5 tentatives max / 15 min par IP
WhatsApp Business / SMS (Twilio)	✉ Remplacé	Remplacé par Bot Telegram (même fonctionnalité)
Statistiques avancées	● Partiel	Visible via dashboard, pas de graphique dédié

Taux de couverture fonctionnelle : 14/17 (82%) — dont 2 remplacements stratégiques assumés et 3 fonctionnalités écartées du périmètre

1.2 Bilan Technique

Compétence CDA	Implémentation dans CALENDRIA
Architecture logicielle	API REST Symfony 6.4 + SPA React - séparation stricte back/front
Base de données relationnelle	PostgreSQL 15 + Doctrine ORM + MERISE (MCD→MLD→MPD)
Sécurité applicative	JWT RS256, Argon2id, rate limiting, CORS, OWASP Top 10
Intégration API externe	Google Gemini 2.5 Flash + Retell AI + API Telegram

Compétence CDA	Implémentation dans CALENDRIA
Containerisation	Docker multi-stage (nginx + PHP-FPM), Docker Compose
CI/CD	GitHub Actions : PHPUnit + TypeScript build à chaque push
Déploiement continu	Railway - déploiement automatique depuis GitHub
Tests automatisés	PHPUnit (unitaires + fonctionnels), TypeScript strict
Gestion de version Git	Commits sémantiques, branches feature, tag v1.0

2. Perspectives d'Évolution

Si le projet devait être poursuivi au-delà de la formation, les évolutions prioritaires seraient :

1. **Notifications push (SMS)** : Rappel automatique J-1 via Twilio ou une API alternative open-source - fonctionnalité à fort impact sur la réduction des no-show
2. **Tableau de bord statistiques avancé** : Graphiques de taux de remplissage, analyse des no-show par période, recettes estimées
3. **Système multi-restaurant** : Association d'un restaurateur à plusieurs restaurants - architecture prévue (clé étrangère nullable dès le MVP)
4. **Application mobile React Native** : Réutilisation des composants existants et de l'API REST
5. **WebSockets temps réel** : Synchronisation instantanée du plan de salle entre plusieurs postes simultanément
6. **API Platform avancée** : Exposition complète des endpoints avec documentation OpenAPI/Swagger automatique
7. **Rotation des clés JWT** : Mécanisme de refresh token pour éviter l'invalidation à chaque déploiement

3. Conclusion

Le projet **CALENDRIA** est un projet complet, déployé en production et couvrant **82%** des fonctionnalités initialement prévues. Les deux fonctionnalités remplacées (WhatsApp et SMS Twilio vers Telegram) l'ont été par des solutions équivalentes, plus accessibles et sans coût ; les trois fonctionnalités non livrées relèvent d'arbitrages de périmètre documentés et non d'un échec technique.

Ce projet a permis d'acquérir une expérience concrète sur des sujets professionnellement pertinents :

- Architecture d'API REST sécurisée avec Symfony
- Intégration d'IA générative dans une application métier réelle
- Déploiement continu sur infrastructure cloud
- Gestion de la qualité via CI/CD automatisée
- Méthodologie Agile Scrum adaptée au développement solo

CALENDRIA constitue un produit réel, maintenable et extensible, prêt à être présenté devant le jury et potentiellement commercialisé auprès de restaurateurs indépendants.

BOUGHERARA Safi - Formation CDA - Juillet 2026

Dépôt GitHub : https://github.com/SafiBougherara/projet_fin_annee

Production : <https://frontend-production-8a43.up.railway.app>

Section XVI - Remerciements

Je tiens à exprimer ma sincère gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet.

À mon Formateur

Mes remerciements les plus chaleureux vont à mon formateur référent, dont l'encadrement pédagogique, les retours constructifs et les conseils techniques tout au long des six jalons ont été déterminants. Sa disponibilité et sa rigueur m'ont permis de maintenir le cap face aux difficultés rencontrées, et d'élever la qualité du livrable final.

À l'Équipe Pédagogique

Je remercie l'ensemble de l'équipe pédagogique du centre de formation pour la qualité du cursus CDA proposé, qui m'a fourni les bases solides en développement full-stack, en architecture logicielle et en gestion de projet nécessaires à la réalisation d'un projet de cette envergure.

À la Communauté Open Source

Ce projet n'aurait pas été possible sans les contributions des milliers de développeurs qui maintiennent les outils utilisés : Symfony, React, Doctrine, TypeScript, Docker, et l'ensemble des packages open source intégrés. La richesse de cet écosystème et la qualité de sa documentation constituent un patrimoine collectif exceptionnel.

Aux Auteurs de Documentation

Je remercie les équipes documentation de Symfony, React, et Google pour la qualité et l'exhaustivité des guides officiels, qui ont été mes premières ressources face à chaque obstacle technique.

Note de Contexte

Ce projet a été développé dans un contexte de formation professionnelle, avec un commanditaire fictif représentant un cas d'usage réaliste. Toutes les données utilisées pendant le développement étaient fictives, en conformité avec les exigences RGPD et éthiques.

Section XVII - Annexes

A. Glossaire

Terme	Définition
API REST	Architecture d'API basée sur HTTP et les verbes REST (GET, POST, PUT, DELETE)
CDCF	Cahier des Charges Fonctionnel
CDA	Concepteur Développeur d'Applications - certification RNCP Niveau 6
CI/CD	Continuous Integration / Continuous Deployment (intégration et déploiement continus)
Couverts	Nombre de personnes pour une réservation
Doctrine	ORM (Object-Relational Mapping) pour PHP/Symfony
Gemini	Modèle d'IA générative de Google (utilisé pour le NLP du chatbot)
JWT	JSON Web Token (système d'authentification par tokens)
MCD/MLD/MPD	Modèle Conceptuel/Logique/Physique de Données (méthode MERISE)
MERISE	Méthode française de modélisation des systèmes d'information
MVP	Minimum Viable Product (produit minimum viable)
NLP	Natural Language Processing - traitement automatique du langage naturel
No-show	Client qui ne se présente pas à sa réservation sans prévenir
ORM	Object-Relational Mapping (mapping objet-relationnel)
OWASP	Open Web Application Security Project
PaaS	Platform as a Service (Railway est un PaaS)
Railway	Plateforme cloud de déploiement Docker (hébergement de CALENDRIA)
Retell AI	Plateforme d'agents vocaux IA utilisée pour le canal téléphonique
RGPD	Règlement Général sur la Protection des Données
RSA	Algorithme de chiffrement asymétrique utilisé pour les clés JWT
Service	Période de service (midi ou soir) dans un restaurant
SPA	Single Page Application (application web monopage)
Telegram Bot	Robot conversationnel sur la plateforme Telegram
Webhook	URL appelée automatiquement par un service externe lors d'un événement

B. Références Réglementaires

RGPD

- [Texte officiel RGPD - CNIL](#)
- [Guide RGPD du développeur - CNIL](#)

Sécurité

- [OWASP Top 10](#)
- [Symfony Security Best Practices](#)

C. Ressources Techniques

Documentation Frameworks

- [Symfony Documentation](#)
- [React Documentation](#)
- [API Platform](#)
- [Doctrine ORM](#)

Documentation APIs Utilisées

- [Google Gemini API](#)
- [Telegram Bot API](#)
- [Retell AI Documentation](#)

Outils de Développement

Catégorie	Outil	Usage
IDE	VS Code / PhpStorm	Développement PHP/TypeScript
Maquettes	Figma	Design UI/UX
Diagrammes	PlantUML	UML, MCD/MLD, architecture
BDD	pgAdmin / DBeaver	Gestion PostgreSQL
API Testing	Postman	Tests d'API REST
Versioning	Git + GitHub	Contrôle de version
CI/CD	GitHub Actions	Automatisation tests/déploiement
Cloud	Railway	Hébergement production

D. Liens du Projet

Ressource	URL
Application en production	https://frontend-production-8a43.up.railway.app
API Backend	https://backend-production-dd10b.up.railway.app
Health Check	https://backend-production-dd10b.up.railway.app/api/health
Dépôt GitHub	https://github.com/SafiBougherara/projet_fin_annee
Badge CI	GitHub Actions - ci.yml

E. Identifiants de Démo

Champ	Valeur
Email	admin@calendria.com
Mot de passe	password123

⚠ Ces identifiants sont destinés uniquement à la démonstration. Ils ne donnent accès qu'aux données fictives du restaurant de démo.

BOUGHERARA Safi - Formation CDA - Juillet 2026

CALENDRIA v2 - Rapport Final restructuré

Formation CDA - Projet Fil Rouge - 2025-2026